Knowledge Transfer — structured-pruning

For anyone picking up this codebase — including future-you six months from now. Read this before changing `prunelib/`.

1. What this is and why it exists — and what it *isn't*, since this changed

Scope note, read this first if you haven't looked at this repo in a while: this package started as a full replacement for the buggy CNN pruning scripts in `pruning_framwork_v4`. Those scripts were subsequently fixed *directly, in that repo*, with more complete method coverage (K-means, SVD, hybrid sequencing) than this package's `prunelib` has. **`pruning_framwork_v4` is now canonical for CNN pruning.** This package's job is narrower: the Transformer-extension experiments, a tested reference implementation of the mask-then-compress design, and — added 2026-09-22/23 — architecture-agnostic `torch.fx` dependency-graph pruning (`graph.py`) plus post-training quantization (`quantization.py`), and — added 2026-09-30 at the author's request — K-Means clustering-based channel selection (`clustering.py`), so the cluster-then-prune-lowest-L1 rule is usable with `graph.py`, the VGG helpers, and the attention-head experiments in this package. Section 4 covers all of them. SVD and hybrid sequencing still live only in `pruning_framwork_v4`; add those there, not here.

The history below is why the code looks the way it does; it's not a claim about what this repo currently supersedes.

This package is a reference implementation of the structured-pruning methods from six of my papers (see `PUBLICATIONS.md`), originally built because four earlier repos (`pruning_framwork`, `_v2`, `_v3`, `_v4`) contained real, confirmed bugs — including one in the exact method the papers are named after.

That history matters for how this code is written, not just as trivia. Every non-obvious design choice in `prunelib/` exists because of a specific, verified bug in the old code. If a change looks like it's undoing one of these choices, it's probably reintroducing the bug. Section 3 maps each choice to the defect it prevents.

The short version of the history: `pruning_framwork_v4/facilitate_pruning.py` computed a Max-3 saliency score into a tensor called `channel_norm`, then selected which channels to prune by comparing a *different* tensor, `channel_norm_temp`, which was never updated and still held the plain L1 norm. The score was computed, stored, returned — and silently ignored. Six more defects (D2–D7) were found in the same file and one neighboring one. None of them were exotic — an off-by-one, a top-k tracker that only updates on new maxima, a boolean tensor passed to `torch.norm()`. All are listed in detail in section 3. All are pinned by a test in `tests/`.

The real irony: the *papers* are fine. Max-3 and L1 produce measurably different accuracy curves in the published results, which they couldn't if v4's broken code had generated them. Working Max-3 code existed — it just was never pushed to GitHub. It isn't recoverable; this package's `prunelib` was the reference implementation until `pruning_framwork_v4` got fixed in place — see the scope note above for where things stand now.

Merged 2026-11-12 (`integrate/generic-pruning-quantization`): this branch's generic dependency-graph pruning and quantization were combined with `main`'s `prunelib.transformer` subpackage (calibration-aware FFN and attention-head pruning, merged in from the standalone `transformer_pruning` repo) and the GLUE study harness in `experiments/research/`. The core API is this branch's (`registry.Registry` everywhere);

the two names `main` had published, `register_saliency` and `register_metric`, remain as aliases, and `main`'s CSD scorer is registered as `"csd"`. Transformer work now happens here — see section 4's `prunelib/transformer/` entry.

2. Repository layout

```
prunelib/
    registry.py   (98 lines)   Registry (name or type -> implementation) + resolve_by_type
                               (class -> entry, via the MRO): the shared extension
                               mechanism every pluggable part below uses
    saliency.py   (177 lines)  Max-k / L1 / L2 / random scoring, single dispatch,
                               any [out, in, *kernel] weight (Linear, Conv1d/2d/3d),
                               a scorer's answer checked (one score per output
                               channel); SALIENCY_METHODS registry
    selection.py  (137 lines)  select_prune_indices_by_method: weight + budget ->
                               prune indices, by any scorer or registered selector,
                               the answer checked against the selector contract;
                               select_prune_indices_among: the same, restricted to
                               candidate (unmasked) channels; prune_count: how many
                               channels a fraction means
    clustering.py (194 lines)  K-Means (Manhattan/Euclidean/Cosine) channel selection:
                               prune the lowest-L1 channels within each cluster;
                               registers itself as the "kmeans" selection rule
    distance.py   (66 lines)   DistanceMetric (pairwise distance + matching K-Means
                               centroid); DISTANCE_METRICS registry
    indices.py    (60 lines)   complement_indices, expand_blocks; index sets are CPU
                               long tensors, moved to the data's device to index it
    surgery.py    (315 lines)  slice_conv2d / slice_depthwise_conv2d / slice_linear /
                               slice_batchnorm primitives (each a resized copy of the
                               original: same class, mode, requires_grad);
                               Conv/BN/FFN/attention-head surgery built on them;
                               live_tensor (a masked tensor as of now)
    masking.py    (128 lines)  two-phase mask-then-compress workflow (torch.nn.utils.prune)
    module_rules.py (227 lines) ModuleRule / ChannelRole: how each layer type takes
                               part in DependencyGraph pruning (role, rebuild, where
                               its output channels are in its weight and in its
                               tensors, how to mask it); MODULE_RULES registry
    op_rules.py   (215 lines)  how a prune passes through ops that own no weights --
                               + - * / (squeeze-and-excitation gates too), functional
                               activations/pooling, cat, flatten, shape queries;
                               OP_PROPAGATORS registry
    graph.py      (333 lines)  torch.fx dependency resolution -- DependencyGraph: the
                               trace (refreshed after a prune resizes something; one
                               or several inputs) and get_pruning_group;
                               special_handlers/LeafTracer (attention-block hook)
    propagation.py (354 lines) Propagation: one prune's walk through the trace -- finds
                               every layer it touches (merges/cat/flatten/depthwise),
                               following the channels on whichever dimension holds
                               them; refuses shared modules and weighted modules
                               with no rule
    group.py      (262 lines)  PruningGroup: applies what the walk found -- .prune()
                               (all replacements built before any is swapped in),
                               or .mask()/.commit_and_compress() (two-phase workflow
                               for a whole dependency group); refuses once stale
    oneshot.py    (86 lines)   prune_model(): score, select and prune any traceable
                               model in one call
    quantization.py (222 lines) Post-training Float16 / linear INT8 / Fixed-Point32
                               quantization (thesis Ch. 6.6) -- separate compression
                               stage from pruning, applied after it;
                               QUANTIZATION_METHODS registry
    vgg.py        (207 lines)  VGG wiring: build_vgg16, prune_vgg_layer, mask_vgg_layer,
compress_masked_vgg
                               (every conv layer, including the last -> classifier[0];
                               only build_vgg16 needs torchvision)
    scanners.py   (67 lines)   Pairwise distance matrix + co-activation scanning
    evaluate.py   (87 lines)   Parameter counts, measured latency, estimated size at a bit-width
    transformer/  model-level Transformer pruning, merged from transformer_pruning
```

```
                  (section 4). Layered, lowest first -- each may import only
                  the ones above it, and any of them may import core prunelib:
        models/       adapters (BERT/RoBERTa layout, resolve_adapter), discover_layers
        measurement/  hooks, FFNCalibrator, HeadCalibrator, ActivationRecorder
        analysis/     scorers, NetworkSaliencyScanner, OV-circuit math,
                      JaccardTwinFinder, kmeans_assign
        selection/    FFNContext/HeadContext/Selection, budget rule, FFN + head selectors
        surgery/      rebuild_linear, FFNSurgeon, AttentionSurgeon
        pipeline/     allocation strategies, prune_* entry points
experiments/
    00_demo.py                (73 lines)  full pipeline, seconds, no dependencies beyond torch
    01_vgg_cifar10_sweep.py   (183 lines)  VGG16/CIFAR-10 — three run modes, see section 4
(experiments/)
    02_bert_sst2_sweep.py      (74 lines)  BERT FFN pruning
    03_head_redundancy.py     (108 lines)  attention head distance scan + K-Means head pruning
    04_coactivation.py         (44 lines)  synthetic co-activation demo
    05_ordering.py             (65 lines)  does the CNN ordering result transfer?
    06_generic_pruning.py     (130 lines)  DependencyGraph on a real ResNet-18
    07_quantization.py         (89 lines)  prune_model() + all three quantization methods, one
pipeline
    08_transformer_techniques.py  prunelib.transformer end to end: calibrated global
                                  head + FFN pruning with bias compensation, twin merge
    research/                     the GLUE pruning study for the paper (section 4):
        glue.py, variants.py,     data/metrics/Trainer, the 23 named conditions,
        run_study.py, summarize.py  resumable runner, mean ± std tables + plots
notebooks/
    pruning_study_kaggle_colab.ipynb   runs research/ on Kaggle or Colab
docs/
    RUNNING_EXPERIMENTS.md        how to run the study and what to report
archive/
    legacy_pipeline/        corrected rebuild of pruning_framwork_v4's original
        config.py, data.py, driver scripts. Superseded by direct fixes to that
        model.py, train.py, repo (which has more complete method coverage) --
        pipeline.py         kept here as a tested mask-then-compress example,
                             not a live alternative. Moved under archive/
                             2026-09-21. See LEGACY_PIPELINE_MIGRATION.md.
tests/          191 core tests, one per historical defect or behavior
    transformer/  199 tests for prunelib.transformer, against a BERT-shaped stub
                  model (no HuggingFace needed); 3 tests in all need a CUDA device
```

Total: ~7,400 lines across `prunelib/`, `experiments/` and `tests/` (excluding `archive/`). Small on purpose — every module does one thing.

3. Design decisions, and the defect each one prevents

If you're refactoring `prunelib`, read this table first.

Decision	Defect it prevents	Where
`compute_score()` is the *only* way to get a saliency score; `select_prune_indices()` takes a score tensor, never a weight tensor	D1: score computed one place, selection reads a different, stale tensor	`saliency.py`
`torch.topk` instead of a hand-rolled "track top-3" loop	D2: manual tracker only rotated on a *new* max, silently dropping mid-range values	`saliency.py::max_k_saliency`

Decision	Defect it prevents	Where
Per-kernel top-k summed across input channels in one vectorized op (`.sum(dim=2)` then `.sum(dim=1)`), not an accumulator reset "once per output channel" but mutated inside an inner loop	D3: score accumulated from monotonically-growing partial maxima across input channels — not the intended quantity at all	`saliency.py::max_k_saliency`
Kernel height and width read from their own tensor dimensions (`kh, kw = weight.shape[2], weight.shape[3]`), never reusing one dimension for both	D4: original iterated `range(size[2])` for *both* height and width, dropping columns on non-square kernels	`saliency.py::max_k_saliency`
`prune_ffn_block` and `prune_conv_bn` raise `ValueError` immediately if shapes don't match, before touching any tensor	D5: a shape bug surfaced as a confusing `IndexError` deep inside an unrelated loop instead of at the actual mistake	`surgery.py` , both functions' first lines
Surgery is pure advanced indexing (`weight.index_select(0, keep_idx)`); no manual per-channel copy loop with a hand-maintained destination counter	D6: the old copy loop's destination index never incremented, so every surviving channel wrote to index 0	`surgery.py::_take` , behind every `slice_*` primitive
Same as D6 — no manual loop, so there's no "off-by-one before first use" to have	D7: `fin_new = fin_org` then `fin_new += 1` before the variable was ever read	`surgery.py` (structurally can't recur)
Pruning a VGG layer replaces modules in-place inside `model.features` rather than rebuilding a whole model from a `feature_list` and copying weights in	This *is* D6/D7's root cause, one level up — the old code's entire "reconstruct then copy" pattern is gone, not just patched	`vgg.py::prune_vgg_layer` / `_apply_vgg_surgery` , `vgg.py::compress_masked_vgg`
Masking uses `torch.nn.utils.prune.custom_from_mask` — one call, full mask computed up front — instead of subclassing `BasePruningMethod` with a `compute_mask` that reconstructs the mask from global state on every call	The systemic bug across *every* driver script in old `pruning_framwork_v4` : a `layer_number` global was supposed to tell `compute_mask` which layer it was on, and the line updating it was commented out in all six scripts, so it silently stayed 0 forever (see `GITHUB_AUDIT.md` section 11)	`masking.py::mask_channels`
`mask_vgg_layer` scores and selects only from `surviving_channels` (not-yet-masked channels), never all channels	Without this, an already-masked channel's weight is zero — the lowest possible score under every criterion — so it wins re-selection on every later iteration, and the pruning schedule advances far slower than the requested fraction implies	`selection.py::select_prune_indices_among` , called by `vgg.py::mask_vgg_layer`

Decision	Defect it prevents	Where
"Which indices are *not* in this index tensor" is a boolean mask + `nonzero()`, never `set(idx.tolist())` plus a Python `range()` comprehension	Not a historical defect, but a recurring anti-pattern found and fixed 2026-09-22 in three places (`masking.py::surviving_channels`, `vgg.py::prune_vgg_layer`, `saliency.py::keep_indices`) — same result, but drops to a Python-level loop over every channel instead of one vectorized op. If you write `set(...)` + a `range()` comprehension against a channel/index tensor anywhere in this codebase, that's the pattern to replace — with `indices.complement_indices`, which every one of those call sites (and the experiments) now uses	`indices.py`
Every pluggable part — saliency scorers, selection rules, distance metrics, quantization methods, prunable layer types, the ops a prune passes through — dispatches through a public `registry.Registry` (keyed by name, or — for layer types in `module_rules.MODULE_RULES` and ops in `op_rules.OP_PROPAGATORS` — by class/fx op target, looked up through the MRO); adding one is a `register_*` call from the caller's own code	Not a historical defect: the open/closed refactor of 2026-10-02. Each was a private dict (`_METHODS`, `_MODEL_METHODS`) or an `if`-chain, so extending meant editing the library — and the same three distance metrics were written out twice (`scanners.py`, `clustering.py`), free to drift apart	`registry.py`, and each module's `*_METHODS` / `*_METRICS` / `MODULE_RULES` / `OP_PROPAGATORS`
Exactly one set of primitives builds a pruned replacement module (`surgery.slice_conv2d` / `slice_depthwise_conv2d` / `slice_linear` / `slice_batchnorm`), and each returns a *copy* of the original with only its tensors and channel counts resized (`_resized_copy`): same class, configuration, device, dtype, train/eval mode and `requires_grad`	`graph.py` and `vgg.py` each had their own copy of `prune_conv_bn`'s index-and-copy logic (and of BatchNorm's running-stat copy); every copy rebuilt modules as CPU float32 with `padding_mode='zeros'`, so pruning a CUDA model left CPU layers in it and a `'reflect'` conv silently changed its border behaviour. Once consolidated, they still built a fresh `nn.Conv2d` / `nn.Linear` (Liskov, 2026-10-10): a weight-standardized `Conv2d` subclass silently became a plain conv and computed something else, a frozen layer came back trainable, a layer of an eval-mode model came back in training mode, and a `BatchNorm` subclass with its own constructor crashed. A masked module comes back unmasked with its masked values baked in; a weight computed by weight norm, spectral norm or a parametrization raises instead of being resized	`surgery.py::_resized_copy`
Exactly one function turns a `prune_fraction` into a channel count (`selection.prune_count`); each caller states only its own floor and cap (`min_prune`, `min_keep`, `available`)	`prune_model`, `prune_vgg_layer` and `mask_vgg_layer` each rounded and clamped by hand, three slightly different ways, and `prune_model`'s could round a fraction just under 1 up to every channel — a zero-width layer that still ran, so nothing failed	`selection.py::prune_count`
A `DependencyGraph`'s per-graph `module_rules=` / `op_propagators=` are layered over the *live* global registries (`ChainMap`), not copied from them	Copying took a snapshot: a rule registered globally after the graph was built reached graphs built without overrides but silently not one built with any override, so the same model pruned differently depending on an unrelated argument	`graph.py::DependencyGraph.__init__`

Decision	Defect it prevents	Where
Every selection rule honours the same contract, including "prune all of them": `kmeans_prune_indices` with the default `n_clusters` returns every index when asked for every channel	`"kmeans"` raised there while `"max_k"` / `"l1"` / `"l2"` / `"random"` returned them all, so `mask_vgg_layer` (which may mask a layer down to nothing) crashed on K-Means where the other rules finished	`clustering.py::kmeans_prune_indices`; `test_mask_vgg_layer_stops_gracefully_when_fully_masked` runs every registered method
`PruningGroup.mask()` masks each layer through its `ModuleRule.mask` (default: `masking.mask_channels`), and `.commit_and_compress()` bakes it in through the same rule's `commit_mask` (default: `masking.commit_mask`) — neither calls the masking code itself	`.mask()` zeroed dim 0 of every layer's weight directly: on a `BatchNorm(affine=False)` it crashed inside `torch.ones_like` (and such a layer can't be masked at all — its running mean turns a zeroed channel into a non-zero constant), and a layer type whose output channels aren't dim 0 would have been zeroed on the wrong axis. A rule that masked some other way still had its mask undone by `masking.commit_mask`, which only knows `torch.nn.utils.prune`'s	`module_rules.py::ModuleRule.mask` / `commit_mask`, `group.py::PruningGroup`
A layer's output channels are read through its rule's `ModuleRule.output_weight` (the weight arranged `[out, ...]`; default `module.weight`), both where `get_pruning_group` counts a starting layer's channels and where `prune_model` scores them	Both read `module.weight` directly, so no rule could describe a type whose output channels aren't dim 0 (open/closed, 2026-10-10). On `ConvTranspose2d` (`[in, out, kh, kw]`), keeping 3 of 6 output channels pruned one — the count came out as its 4 *input* channels — with no error	`module_rules.py::ModuleRule.output_weight`; `test_module_rule_says_where_a_layers_output_channels_are`
Every scorer accepts any `[out, in, *kernel]` weight (`saliency._channel_view` flattens the kernel, whatever its rank)	Scorers accepted only 2D (Linear) and 4D (Conv2d) weights, so a Conv1d or Conv3d taught to `DependencyGraph` through a module rule still couldn't go through `prune_model` (open/closed, 2026-10-10)	`saliency.py::_channel_view`
`quantize_model_` quantizes every floating-point parameter each listed module type owns directly, not `weight` and `bias` by name	A listed type that names its parameters otherwise was silently left as it was (`MultiheadAttention`'s `in_proj_weight`), and `nn.LSTM`, whose `bias` is a flag rather than a tensor, crashed (open/closed, 2026-10-10)	`quantization.py::quantize_model_`
A `DependencyGraph` re-traces before a walk when any parameter or buffer has been resized since its last trace, and looks up a module object in the model as it is now	It kept its first trace's shapes and an id → name table, though a prune changes both. After one branch of a `cat` was pruned, the next prune of another branch was offset by the first branch's *old* width and removed the wrong input channels of the layer after the `cat`, silently; and the module a prune swapped in (what `model[2]` returns from then on) was "not a submodule of the traced model" (2026-10-10)	`graph.py::DependencyGraph.get_pruning_group` / `_trace` / `_resolve_name`
`DependencyGraph`'s ShapeProp forward pass runs in eval mode without gradients, every submodule's own mode restored afterwards; the trace itself sees the model's own mode	It ran in whatever mode the model was in, so building a graph on a training-mode model nudged every BatchNorm's running statistics — once per trace, and a graph now re-traces after each prune that resizes something	`graph.py::DependencyGraph._trace`
The walk back to a skip branch's producer passes only through ops that keep channels one-to-one; anything else raises at that op	It stepped through every function call as if indices carried over, so a skip branch built with `cat` pinned the pruned indices on the wrong conv and failed later with a misleading "two branches disagree" error	`graph.py::Propagation.find_producer`

Decision	Defect it prevents	Where
An activation, dropout, pooling or resize written as a function or tensor method passes a prune through when it is registered as channel-wise (`propagate_channelwise` , built in for the common ones); a module passes by its shape. An unregistered function still raises, naming the fix	Functions had no pass-through at all: `nn.ReLU()` and `nn.MaxPool2d` carried a prune on while `F.relu(x)` , `x.relu()` and `F.max_pool2d(x, 2)` raised, so whether a model could be pruned depended on how its activations were spelled (Liskov, 2026-10-10). Functions don't get the modules' shape-based pass-through because a function's shape says too little: `torch.flip(x, [1])` keeps every channel and reorders them	`op_rules.py::propagate_channelwise`
`+ - * /` share one propagator (`propagate_elementwise`), which couples the other operand's producer only when that operand carries the same channels; a scalar or a channel-broadcast operand is passed by	Only `+` was registered (`propagate_add`), and it took every merge to be two same-shape branches: `x + 1` was refused, the 1-channel conv behind a `[N, 1, H, W]` spatial gate was told to lose channels it doesn't have, and `*` had no propagator, so no squeeze-and-excitation block (MobileNetV3, EfficientNet) could be pruned through (2026-10-10)	`op_rules.py::propagate_elementwise` ; `test_mobilenet_v3_prunes_through_its_squeeze_excitation_blocks`
A flatten is carried only when it really is `[N, C, H, W]` → `[N, C·H·W]` ; reading a tensor's shape (`x.size(0)` , `x.shape`) carries nothing on	Every view/reshape/flatten of a 4D tensor had its indices expanded, so after `x.flatten(2)` the walk failed later at the next layer, naming it instead of the reshape; and `x.view(x.size(0), -1)` , the usual classifier flatten, was refused at the `size` call (2026-10-10)	`op_rules.py::propagate_flatten` , `_shape_query`
A module with no rule is passed through by its shape only if it owns no parameters or buffers; one that does raises, naming `register_module_rule` / `register_op_propagator` . One BatchNorm rule covers every `_BatchNorm` and `_InstanceNorm`	A module that kept its channel count was passed through whatever it was (Liskov, 2026-10-11): a `GroupNorm` , a `PReLU(C)` , a `ConvTranspose2d(C, C)` — which mixes channels — and `SyncBatchNorm` (what DDP's `convert_sync_batchnorm` leaves; a sibling of `BatchNorm2d` , so the rule registered for `BatchNorm1d` / `2d` missed it) kept their old width while everything around them shrank, and the model failed on its next forward pass. `vgg_conv_bn_positions` missed `SyncBatchNorm` the same way	`propagation.py::Propagation._visit_module` , `module_rules.py` registrations, `vgg.py::vgg_conv_bn_positions`
A module called more than once in `forward()` can't be pruned: the walk refuses to record a target on it	The graph kept one node per module — the last call — so only that call's consumers were walked; the other call's consumers (and its own input) kept the old width	`propagation.py::Propagation._check_single_call`
A layer's rule says which dimension of its tensors holds its channels (`ModuleRule.channel_dim` : 1 by default, -1 for a Linear), and the walk follows them there, counted from the end so they line up across broadcasting; a layer reading another dimension raises	Channels were dim 1 everywhere (open/closed, 2026-10-11). For a Linear over `[N, T, features]` that's `T` : a `[N, T, 1]` gate looked like it carried the pruned features (its 1-output Linear was told to lose them), a per-feature `[features]` parameter looked like it carried none (kept its width while the Linear shrank), and a Linear over a conv output's *width* had its inputs pruned by the conv's channel indices	`module_rules.py::ModuleRule.channel_dim` , `propagation.py::Propagation.channel_dim` , every propagator in `op_rules.py`
Each pruned input of a `cat` is one *part* of what its output loses (`Propagation.forward_part`); the walk carries all parts on together once nothing else can add one, and counts every place a tensor appears	Each branch called `forward` on the `cat` , and the walk skips a node it has already left: with two pruned branches (`cat([y, relu(y)])` , GhostNet's `cat([x, cheap(x)])`) only the first one's channels left the layer after the `cat` , and `cat([y, y])` lost the second copy's	`op_rules.py::propagate_cat` , `propagation.py::Propagation.forward_part` / `_run`

Decision	Defect it prevents	Where
`PruningGroup.prune()` builds every replacement before swapping any in, and every `PruningGroup` method refuses once the model has been resized since the group was computed (`group.tensor_layout`, which reads masking as no change); the target maps are read-only views	A refused rebuild (a weight-normed conv) left the layers rebuilt before it swapped in — a half-pruned model. `.prune()` twice pruned the same positions again (8 channels down to 4 for a group that keeps 6), and two groups computed up front for two `cat` branches removed the wrong input of the layer after the `cat` with the second one — both silently. Writing into `group.output_targets` skipped the disagreement check	`group.py::PruningGroup`
A masked layer's tensors are read as `weight_orig * weight_mask` now (`surgery.live_tensor`), wherever pruning reads a weight: the slicing primitives, `ModuleRule.output_weight`, `vgg.mask_vgg_layer` / `prune_vgg_layer`	`torch.nn.utils.prune` recomputes `weight` only in a forward pre-hook, so after an optimizer step it is one step stale. `.prune()` on a masked group rebuilt the layer as it was before the last step, and `prune_model` and the next `mask_vgg_layer` iteration scored those stale values	`surgery.py::live_tensor`
A scorer's answer must be one score per output channel, a selector's exactly `prune_amount` distinct in-range channels, a quantization method's the parameter's own shape — checked where each is dispatched, naming the method	Plugins were trusted (Liskov, 2026-10-11): a scorer reducing over the wrong axis had its 4 per-input-channel scores ranked as 8 output channels', a selector returning a channel twice pruned under budget, and a quantization method returning `[out, 1]` was broadcast over every row — all silently	`saliency.py::compute_score`, `selection.py::_checked_selection`, `quantization.py::quantize_model_`
`ModuleRule.mask`'s default refuses for a rule that overrides `output_weight` without overriding `mask`	The default zeroes dim 0 of the weight, so the `ConvTranspose2d` rule (weight `[in, out, kh, kw]`) had its *input* channels zeroed by `PruningGroup.mask()`	`module_rules.py::ModuleRule.mask`
The walk back along a skip branch follows an elementwise merge through whichever operands carry the pruned channels	It followed a merge only if every operand had the same channel count, so a skip branch gated by a `[N, 1, H, W]` map was refused	`propagation.py::Propagation.find_producer`
`quantize_model_` quantizes every floating-point parameter by default, each once (`module_types=` narrows it; `QUANTIZABLE_MODULE_TYPES` is the thesis's scope)	The default was a closed type list, so a Conv1d, LayerNorm, Embedding or `SyncBatchNorm` stayed at full precision silently — the measured drift under-reported, and `estimate_size_bytes(model, 16)` counted them as 16-bit anyway. A weight tied between two listed modules was quantized twice	`quantization.py::quantize_model_`
`get_pruning_group` refuses a `keep_idx` that keeps no channel; `prune_model` refuses a `dependency_graph` traced from another model; `example_input` may be a tuple (one per positional input) for `DependencyGraph`, `prune_model` and `measure_latency`	An empty `keep_idx` left a zero-width Linear that still ran; given a layer by name, `prune_model` scored it in `model` and pruned the graph's model with those scores; a model with two inputs couldn't be traced or timed	`graph.py::DependencyGraph`, `oneshot.py::prune_model`, `evaluate.py::measure_latency`

Decision	Defect it prevents	Where
Index sets are CPU `long` tensors wherever their weights are (`indices.as_index`); every indexing site moves them to the data's device (`indices.on_device_of`); K-Means runs on the CPU; `measure_latency` synchronizes a CUDA device before reading the clock	Indices landed on whichever device made them, so on a GPU the first `complement_indices` (a CPU mask indexed by CUDA indices) or `index_select` (CUDA weight, CPU indices) failed, and K-Means' CPU generator failed on CUDA weights. Fixed 2026-10-11 by reasoning, **verified on a GPU 2026-10-13** (GTX 1650, `venv-cuda/`): `test_a_cuda_model_prunes_with_every_selection_rule` passes. Every factory call that builds an index set, a mask over one, or a CPU-generator draw also says `device="cpu"` (2026-10-13): left implicit, they followed torch's default device, so under `with torch.device("cuda"):` index sets came back on the GPU and `random` / `kmeans` failed (`test_cuda_as_the_default_device_still_keeps_index_sets_on_the_cpu`)	`indices.py` , every `slice_*` , `clustering.py` , `saliency.py` , `evaluate.py`
`prunelib.transformer` builds every resized Linear through `rebuild_linear` (dtype, device, `requires_grad` preserved)	A bare `nn.Linear` converted an fp16 FFN to fp32 and un-froze frozen layers during a "heal"	`transformer/surgery/linear_ops.py`
Transformer budgets `round` , never truncate (`keep_count`)	`int()` always pruned one unit more than asked — a one-directional bias compounding over a whole-model sweep	`transformer/selection/budget.py`
Every calibration/recording hook drops padding through one `flatten_tokens`	Counting `[PAD]` positions dragged means toward the pad output and made every neuron look co-active with every other	`transformer/measurement/hooks.py`
Twin detection uses true Jaccard (`intersection / union`), and twin chains resolve by union-find onto the lowest index	The original notebook divided by `fires_A + fires_B` ; pair-order-dependent resolution merged a neuron *into an already-removed neuron*	`transformer/analysis/redundancy.py` , `transformer/selection/ffn_selectors.py::TwinRedundancySelector`

If you ever find yourself writing a loop that tracks a running max/min by hand, or a loop with a manually-incremented destination index into a tensor — stop. Both patterns have already caused a real bug in this project's history. Use `torch.topk` / `torch.sort` and advanced indexing instead.

4. Module walkthrough

`prunelib/saliency.py`

Five scoring functions (`max_k_saliency` , `l1_saliency` , `l2_saliency` , `csd_saliency` , `random_saliency`), all with signature `(weight) -> [out]` for any weight laid out `[out, in, *kernel]` — a Conv2d `[out,in,kh,kw]` , a Linear `[out,in]` , a Conv1d or Conv3d (`_channel_view` reshapes each to `(out, in, spatial)` ; it used to accept 2D and 4D only, so a Conv1d taught to the graph by a module rule couldn't be scored by `prune_model`), routed through `compute_score(weight, method=...)` , which checks the answer is one score per output channel and raises naming the method otherwise (it used to rank whatever a registered scorer returned). `select_prune_indices` and `keep_indices` operate on the *score* tensor, not the weight —

this separation is what makes D1 structurally impossible to reintroduce, since there's no code path where a caller has both a weight tensor and a stale score tensor in scope at the same time. The scorers are entries in `SALIENCY_METHODS` (registered with `@register_saliency_method("max_k")` etc.), so a new one added from outside the package is immediately valid everywhere `method=` is accepted — see "Extension points" below.

`max_k_saliency` is the one to understand well: it flattens each kernel to 1D, takes the top-`k` magnitudes with `torch.topk`, sums them (`kernel_scores`, shape `[out, in]`), then sums across the input-channel axis exactly once (`channel_scores`, shape `[out]`). That's the whole algorithm. If a change here doesn't fit in three lines of einsum-shaped reasoning, it's probably reintroducing a shape bug — this function has been the site of 4 of the 7 historical defects.

`csd_saliency` (registered as `"csd"`, from `main` in the 2026-11-12 merge) is the L1 dispersion of each unit's incoming weights from their own mean, adapted from the "Enhancing Deep Compression of CNNs" paper's CSD regularizer as a one-shot score. `prunelib.transformer.CSDScorer` calls this same function, so CSD is one implementation for CNN channels and FFN neurons.

`prunelib/surgery.py`

Four primitives — `slice_conv2d(conv, keep_out=, keep_in=)`, `slice_depthwise_conv2d(conv, keep)`, `slice_linear(linear, keep_out=, keep_in=)`, `slice_batchnorm(bn, keep)` — and three seam-aware functions built on them: `prune_conv_bn` (Conv2d + optional BatchNorm2d + optional next Conv2d), `prune_ffn_block` (Linear → activation → Linear), and `prune_attention_heads`. Everything builds *new* modules of the correct smaller size and copies the kept rows/columns in — never mutating the originals. The seam-aware functions validate the seam (that the tensor shapes on either side of the cut actually match) before doing anything, and raise `ValueError` with a specific message if not.

The primitives are the only code in `prunelib` that constructs a pruned module — `graph.py` (through `module_rules.py`) and `vgg.py` call them too — so "a replacement is a drop-in substitute" is enforced in one place (`_resized_copy`): each returns a *copy* of the original with only the sliced tensors and channel counts replaced — same class (a weight- standardized `Conv2d` subclass stays one), kernel/stride/padding/dilation, `padding_mode`, device, dtype, train/eval mode and `requires_grad`. Until 2026-10-10 they built a fresh `nn.Conv2d`/`nn.Linear` instead, which silently turned a subclass into its base class, unfroze frozen layers and put an eval-mode model's rebuilt layers back in training mode. A module masked with `torch.nn.utils.prune` comes back as a plain module holding the masked values; one whose weight is *computed* (weight norm, spectral norm, a `parametrize` parametrization) raises `NotImplementedError` rather than be resized and recomputed at its old size. `slice_conv2d` raises for a grouped conv rather than building `groups=1` (which, for a depthwise conv, used to silently broadcast each single-channel filter across every input channel).

A masked layer is sliced as it is *now*: `live_tensor(module, name)` works out `weight_orig * weight_mask` instead of reading the `weight` attribute, which `torch.nn.utils.prune` refreshes only in a forward pre-hook and which is therefore one optimizer step stale while you fine-tune with a mask. Everything in `prunelib` that reads a possibly-masked weight goes through it (`ModuleRule.output_weight`'s default, `vgg.py`). Indices may arrive on any device; each primitive moves them to the tensor it indexes (see `indices.py` under "Extension points").

One deliberate design choice worth knowing: `prune_conv_bn` copies BatchNorm's running mean/var across to the new, smaller BN layer. The original code didn't do this at all — pruned models ran without error but produced silently wrong activations until the next full training pass reset the running stats. This isn't "more correct" in some abstract sense, it's specifically there because that failure mode doesn't throw, so nothing would tell you it happened.

`prunelib/masking.py`

The two-phase mask-then-compress workflow: `mask_channels` zeroes a channel's weight *and* bias via `torch.nn.utils.prune.custom_from_mask` (non-destructive — shapes don't change, and it composes correctly across repeated calls on the same layer, which is what makes it safe to call once per pruning iteration). `commit_mask` bakes the zeros in permanently. `compress_masked_conv_bn` reads back which channels ended up all-zero and calls `surgery.prune_conv_bn` on the survivors — same tested surgery code as everywhere else, just fed indices derived from a committed mask instead of a fresh score.

The point of this two-phase split: you can fine-tune or evaluate with the mask active, across as many iterations as you want, before committing to an architecture change. Gradients naturally don't reach masked positions (`weight = weight_orig * mask`, so `d(loss)/d(weight_orig) = d(loss)/d(weight) * mask`, which is zero wherever the mask is zero) — no extra bookkeeping needed to keep pruned channels from "coming back" during fine-tuning.

`tests/test_masking.py::test_compress_masked_conv_bn_matches_direct_surgery` is the test to read if you're unsure this is equivalent to just calling `prune_conv_bn` directly — it proves the two paths produce numerically identical output.

`prunelib/graph.py` (with `propagation.py`, `op_rules.py`, `group.py`, `oneshot.py`)

Split 2026-10-10 into four modules with one job each, and a fifth on 2026-10-11: `graph.py` *traces* the model, `propagation.py` *follows* one prune decision through the trace (`Propagation`), `op_rules.py` says how a prune passes through add/cat/flatten, `group.py` *applies* the result (`PruningGroup`), and `oneshot.py` *decides* what to prune (`prune_model`). Before the first split, `graph.py` was 671 lines and the dependency tracer imported the selection rules only `prune_model` needed; the walk moved out next because it changes far more often than the trace (six of the 2026-10-11 fixes were to the walk, two to the trace). Everything below still holds; only the module each piece lives in changed. All five are re-exported from `prunelib`, so `from prunelib import DependencyGraph, PruningGroup, Propagation, prune_model` is unchanged.

Added 2026-09-22 to close a gap every other module in `prunelib` leaves open: `prune_conv_bn` needs `next_conv=` passed in by hand, `vgg.py` only works because it hardcodes that VGG's `.features` is a flat `Sequential` — nothing before this traced an arbitrary `nn.Module` and worked out its own wiring. `DependencyGraph(model, example_input)` traces the model with `torch.fx` (again only when a prune has resized something, see below); `get_pruning_group(layer, keep_idx)` walks forward from a prune decision to find every other layer it couples to — a residual/skip partner (via elementwise add, walked backward to whichever conv/Linear actually produced the other branch), a squeeze-and-excitation gate (`x * scale`, the same walk: `scale`'s producing conv loses the same output channels), a `torch.cat` (indices get shifted by the other branches' channel counts), the conv-output-flattened-into-Linear classifier boundary (channel indices expand by the spatial size), and depthwise convs (indices pass straight through, since input and output channel count are the same dimension there). Anything it doesn't recognize — a non-depthwise grouped conv, an attention block, a module that owns weights but has no rule, a module called more than once — raises `NotImplementedError` rather than silently mis-wiring something; see the module's own docstring for the full list of what's deliberately out of scope (no automatic attention-block discovery; nothing that moves channels to another dimension, like a `permute`; a few other named limitations). Several pruned branches of one `cat` are carried on together: each is a *part* of what the `cat` loses (`Propagation.forward_part`), merged once nothing else in the walk can add one. Until 2026-10-11 only the first branch to reach the `cat` got through. `example_input` may be a tuple, one tensor per positional input of `forward`.

Which layer types it understands lives in `module_rules.py`, not in `graph.py`: each type's `ModuleRule` answers `role(module)` — a `ChannelRole.MIXING` layer (ungrouped Conv2d, Linear) is where a prune *stops* (only its input shrinks) and where channels are *produced* (a starting layer, or the far end of a skip connection); a `ChannelRole.PER_CHANNEL` layer (BatchNorm, depthwise conv) shrinks and passes the prune through — `rebuild(module, prune_out, prune_in)`, and three optional ones whose defaults assume output channels are dim 0 of `weight` (true of every built-in type, not of `ConvTranspose2d`'s `[in, out, kh, kw]`): `output_weight(module)` (the weight arranged `[out, in, *kernel]`, which a starting layer's channel count and `prune_model`'s scores are read from), `mask(module, prune_out)` (zero those output channels for `PruningGroup.mask()`; the default refuses for a rule that overrides `output_weight` but not `mask`, since it would zero dim 0 anyway) and `commit_mask(module)` (bake that in for `.commit_and_compress()`). A fourth, `channel_dim(module)`, says which dimension of the tensors the layer reads and writes holds its channels — 1 by default, -1 for a Linear — and the walk follows the channels there, counted from the end (`Propagation.channel_dim(node)`) so they line up across broadcasting; a layer that reads its channels from a different dimension than the one they're on raises. Until 2026-10-11 it was dim 1 everywhere, which for a Linear over `[N, T, features]` is `T`. The forward walk, the backward skip-connection walk, `PruningGroup.prune()` / `.mask()` / `.commit_and_compress()` and `prune_model` depend only on those answers; `test_extension_points.py::test_module_rule_says_where_a_layers_output_channels_are` writes a `ConvTranspose2d` rule. The methods are positional-only, so a rule can name its parameters for its own type (`role(self, conv)`) without breaking the base contract. Register a rule for another type (`register_module_rule`, or `module_rules=` on one `DependencyGraph`) and every path handles it; `tests/test_extension_points.py` does this for Conv1d. Which *ops* it understands is the same idea one level over: each is an op propagator in `op_rules.OP_PROPAGATORS` — `propagate_elementwise` (`+ - * /`; it couples the other operand only when that operand carries the same channels, so `x * 0.5` and a `[N, 1, H, W]` spatial gate pass by), `propagate_channelwise` (activations, dropout, pooling and resizing written as functions or tensor methods; a module form like `nn.ReLU` needs no entry), `propagate_cat`, `propagate_flatten` (only a true `[N, C, H, W]` → `[N, C·H·W]`, via `torch.flatten`, a tensor method or `nn.Flatten`), and a no-op for shape queries (`x.size(0)`) — each a function `(walk, source, node, idx)` that sees the walk only through `Propagation`'s seven methods (`shape`, `channel_dim`, `add_output_target` / `add_input_target`, `forward`, `forward_part`, `find_producer`). The graph and the `PruningGroup` being filled stay private; a propagator used to be handed both, and could call `group.prune()` halfway through a walk. The backward walk (`find_producer`) only passes through ops that keep channels one-to-one — a tensor-free module that keeps the channel count, a function registered with `propagate_channelwise`, or one registered with `propagate_elementwise`, through whichever operands carry the channels (so a skip branch gated by a `[N, 1, H, W]` map now leads back to the gated tensor's producer; it used to be refused) — so a skip branch built with `cat` or `chunk` raises at that op instead of pinning indices on the wrong conv. A module with no rule is passed through by its shape only if it owns no parameters or buffers: a `GroupNorm`, `PReLU(C)` or `ConvTranspose2d(C, C)` used to be passed through because it kept its channel count, and kept its old width while everything around it shrank. `DependencyGraph` holds what's fixed for the model as it currently is — the trace and every node's recorded shape — and re-traces before a walk when a prune (through it or any other route) has resized a parameter or buffer since; a layer passed as a module object is looked up in the live model. One graph therefore covers a whole pruning session. It used to keep its first trace's shapes, so after one branch of a `cat` shrank, pruning another branch offset its indices by the first branch's old width (the wrong input channels downstream, no error), and a module an earlier prune swapped in wasn't recognised. ShapeProp's forward pass runs in eval mode without gradients (modes restored after), so tracing no longer nudges BatchNorm running statistics; the trace itself still sees the model's own mode. `Propagation` holds one `get_pruning_group` call's state (the group being filled, the nodes visited, where each tensor's channels are, the `cat` parts waiting), which is why none of it is threaded through every method.
`PruningGroup.add_output_target` / `add_input_target` own the "two branches disagree about one layer" check; the target maps themselves are read-only views. The starting layer must be `MIXING`: starting from a

depthwise conv used to be accepted and leave its producer's output wider than its new input (a model that failed on the next forward pass); it now raises `TypeError`. A `keep_idx` that keeps no channel raises `ValueError`, and a module called more than once in `forward()` (shared weights) can't be pruned at all.

A `PruningGroup` is computed against the model as it is at that moment, so it refuses (`RuntimeError`) once a parameter or buffer has been resized since (`group.tensor_layout`, which reads masking as no change): apply a group before computing the next. `.prune()` used to be happy to run twice, and a group computed before another group's prune removed channels by their old numbers. `.prune()` also builds every replacement before swapping any in, so a layer whose rebuild refuses (a weight-normed conv) leaves the model as it was rather than half-pruned.

One real deviation from the rest of this library, called out in `group.py`'s docstring: `PruningGroup.prune()` mutates the model in place instead of returning new modules for the caller to wire up. That's deliberate — the whole point of dependency-graph pruning is that the caller shouldn't need to know the graph shape well enough to do that wiring themselves.

`experiments/06_generic_pruning.py --tiny-check` is the test to read if you want proof this holds on something real: it prunes a conv inside a `torchvision.models.resnet18` `BasicBlock` with no seam passed by hand, and the resulting `PruningGroup` correctly cascades through the rest of that residual stage (both blocks share the same channel count) and into the next stage's transition convs — none of that cascading was specifically designed for, it falls out of the forward-walk naturally.

Added 2026-09-23, three extensions, none changing the behavior above:

- `prune_model(model, example_input, layer, prune_fraction, method=...)` (`oneshot.py`) — the `vgg.py::prune_vgg_layer` convenience (score → select → surgery, one call), generalized to any model `DependencyGraph` can trace instead of hardcoding VGG's flat `.features`. At least one channel always survives (`selection.prune_count`). A `dependency_graph=` traced from a different model raises. `experiments/07_quantization.py` uses it as its pruning step; `test_graph.py::test_prune_model_scores_and_prunes_in_one_call` is the unit test.
- `PruningGroup.mask()` / `.commit_and_compress()` — the two-phase mask-then-compress workflow from `masking.py` (see its entry above for it), extended to a whole dependency group: `.mask()` zeroes every output-target module through its `ModuleRule.mask` instead of immediately rebuilding, so you can fine-tune with the group's channels masked before committing; `.commit_and_compress()` bakes the zeros in and runs the same rebuild `.prune()` does. `test_graph.py::test_mask_then_commit_and_compress_matches_direct_prune` proves the two paths produce identical weights, mirroring `test_masking.py::test_compress_masked_conv_bn_matches_direct_surgery`'s reasoning one level up. `.mask()` raises `NotImplementedError` for a group built by a special handler (below): a handler rebuilds its block in one step, so there is nothing to mask, and `.mask()` used to skip it silently while the caller fine-tuned as if it had been masked.
- `special_handlers` (a `{type: handler}` map on `DependencyGraph.__init__`) and `LeafTracer` — the extension point for attention-block pruning this module's docstring disclaims doing automatically. When `get_pruning_group`'s *starting* layer is an instance of a registered type, the handler (not the Conv2d/Linear/BatchNorm logic) decides how to rebuild it — see `test_graph.py::test_special_handler_prunes_attention_block_via_prune_attention_heads` for a worked example wiring a handler to `surgery.prune_attention_heads`. Subclasses of a registered type are matched too (MRO lookup, the same `isinstance` semantics `LeafTracer` already used — it used to be an exact-`type()` match, so a subclass traced as a leaf and then fell through to "expected Conv2d or Linear"). `LeafTracer` exists because a custom block class isn't a leaf under `fx.Tracer`'s

default rule, so without it `DependencyGraph` would trace straight through the block and never see it as one node to hand to the handler. Deliberately scoped to the *starting* layer only — a block encountered mid-propagation (as a consequence of some other layer's prune) still raises `NotImplementedError`, same as before; translating an arbitrary upstream channel prune into a head-pruning decision is a different, harder problem this hook doesn't attempt.

`prunelib/quantization.py`

Added 2026-09-23 to close the gap section 10.2 (below) used to describe in full: quantization wasn't implemented anywhere in this codebase, despite being half the thesis's title. Three post-training methods, reference implementations of thesis Ch. 6.6: `quantize_float16` / `dequantize_float16` (direct cast, the thesis's best accuracy/memory tradeoff), `quantize_int8_linear` / `dequantize_int8_linear` (the standard scale/zero-point affine mapping, Eq. 6.7–6.10, returning an `Int8Tensor` struct since realizing INT8's actual 4x memory reduction means storing `int8` values plus scale/zero-point rather than a same-shape float tensor), and `quantize_fixed_point32` (1-sign/3- integer/28-fractional bit format, thesis's own finding that it's markedly worse than Float16 since a fixed exponent can't adapt to a layer's actual weight distribution — kept for reproducing that comparison, not as a recommendation). `quantize_model_` applies Float16 or Fixed-Point32 to every floating-point parameter in the model, each once (a weight tied between two modules isn't quantized twice), or — with `module_types=` — to those the listed types own directly (`QUANTIZABLE_MODULE_TYPES` is the thesis's Linear/Conv2d/BatchNorm scope). The default used to be that closed list, so a Conv1d, LayerNorm, Embedding or `SyncBatchNorm` stayed at full precision without a word; and it used to look up `weight` / `bias` by name, so `MultiheadAttention`'s `in_proj_weight` was silently skipped and `nn.LSTM` crashed. A method's answer must keep the parameter's shape (it used to be broadcast back). It works in place — the one function in this module (besides `masking.commit_mask`, its precedent) that deviates from `prunelib`'s "never mutate what's passed in" convention, since quantization changes no tensor's shape, so there's no smaller module to construct and return instead. `evaluate.estimate_size_bytes(module, bits_per_param)` complements it — a quick way to compare precision options' memory footprint without writing a checkpoint to disk for each, the helper section 10.2 used to say `evaluate.py` was the natural home for. `experiments/07_quantization.py` runs the whole pipeline (`prune_model` then all three methods) end to end; `tests/test_quantization.py` is the unit suite, including a brute-force comparison against the Eq. 6.7–6.9 formula (`test_int8_quantize_matches_the_documented_formula`) in the same style as `saliency.py`'s brute-force tests.

`prunelib/vgg.py`

VGG-specific wiring on top of the layer-level primitives above. `build_vgg16` wraps `torchvision.models.vgg16` (using `weights=`, not the deprecated `pretrained=True/False`). `prune_vgg_layer` is the direct, one-shot version (score → select → surgery, immediately). `mask_vgg_layer` / `compress_masked_vgg` are the two-phase version — see the design-decision table in section 3 for the correctness fix `mask_vgg_layer` needed (excluding already-masked channels from re-selection).

Every conv layer can be pruned, including the last one (added 2026-09-30): it feeds `classifier[0]`, a `Linear` over the flattened `[C, 7, 7]` avgpool output, so pruning channel `c` also drops that Linear's input columns `[c*49, (c+1)*49)` (`_shrink_classifier_input`, the VGG- specific version of `op_rules.propagate_flatten`). Both `prune_vgg_layer` and `mask_vgg_layer` take any `method=` that `selection.select_prune_indices_by_method` accepts, including `"kmeans"` or a selector callable. Only `build_vgg16` needs torchvision (imported inside it), so the surgery functions work on any torchvision-shaped VGG without the optional dependency installed. Any BatchNorm after a conv is pruned with it, `SyncBatchNorm` included (only `BatchNorm2d` was, so a VGG converted for DDP training was pruned without

its BatchNorms), and a masked conv is scored by its live weights (`surgery.live_tensor`), not the ones the last forward pass left.

`archive/legacy_pipeline/`

Moved under `archive/` 2026-09-21 (was `legacy_pipeline/` at the repo root; import as `archive.legacy_pipeline`) to make explicit that this package is `prunelib` plus the Transformer experiments first, not a pipeline you run in place — see README.md's opening line. A corrected rebuild of `pruning_framwork_v4`'s original six driver scripts, done in parallel with this package, before those scripts got fixed directly in that repo. **Now superseded**: `pruning_framwork_v4`'s own fix has more complete method coverage (K-means, SVD, hybrid sequencing) than `legacy_pipeline` does, since `legacy_pipeline` only offers this package's selection rules. Its `--method` choices come from `selection.available_methods()`, so `"kmeans"` and any rule registered later show up there with their default arguments (`metric="manhattan"` for K-Means); nothing else about the frozen pipeline changes. Don't extend `legacy_pipeline` to add parity with `pruning_framwork_v4` — that work belongs in `pruning_framwork_v4` itself. What's still worth reading here: `pipeline.py::run_pruning` is a complete, tested example of the mask-then-compress workflow applied to a whole VGG16 end to end (mask every prunable layer per iteration → fine-tune → evaluate → repeat → compress once at the end), and `tests/test_legacy_pipeline.py` is the regression test for that. See `LEGACY_PIPELINE_MIGRATION.md` for the full defect-by-defect mapping from the original scripts.

`prunelib/scanners.py`

`pairwise_distance_matrix(vectors, metric)` computes Manhattan/Euclidean/ Cosine — the three metrics compared in Paper 2 — via `distance.py` (below), the same metric definitions `clustering.kmeans` uses, so a metric registered with `register_distance_metric` works here too. It computes distances only; the clustering and selection live in `clustering.py` (below).

`CoActivationScanner` computes pairwise Jaccard similarity on boolean firing masks, with a `firing_rate_ceiling` (default 0.9) that excludes near- universal-firing units before computing similarity. This exists because a unit that fires on 95%+ of tokens will show spuriously high overlap with *any* other frequently-firing unit — that's an artifact of both units being non-selective, not evidence of redundancy. This hasn't been validated against real model activations yet (see section 6) — the logic is tested against synthetic masks in `tests/test_scanners.py` and demonstrated in `experiments/04_coactivation.py`, but nobody has confirmed the 0.9 default is the right threshold on an actual fine-tuned model.

`prunelib/clustering.py`

Added 2026-09-30. `kmeans(vectors, n_clusters, metric)` is plain-torch K-Means (no scikit-learn dependency) with K-Means++ seeding. Each metric uses the centroid update that actually minimizes it: mean for Euclidean, **coordinate-wise median for Manhattan** (K-Medians; the mean minimizes squared L2, not L1), and normalized mean for Cosine (spherical K-Means). Empty clusters are re-seeded, so every cluster always has at least one member. It runs on the CPU whatever device the weights are on, so the same seed clusters the same way everywhere (on a CUDA weight it used to fail at its CPU generator).

`kmeans_prune_indices(weight, prune_amount, n_clusters=None, metric=...)` is the paper's selection rule. It clusters output channels on their flattened weights and prunes the **lowest-L1 channels within each cluster**, keeping each cluster's highest-L1 member as its representative. The default is `n_clusters = out - prune_amount`, which leaves exactly one survivor per cluster. A smaller `n_clusters` still protects one representative per cluster and prunes the lowest-L1 channels among the rest. It raises rather than empty a cluster. Asked to prune every channel with the default `n_clusters`, it returns them all, as every other

selection rule does (there is no cluster left to protect); it used to raise, so `mask_vgg_layer` crashed on `"kmeans"` where `"max_k"` masked a layer down to nothing.

`kmeans_prune_indices` registers itself as the `"kmeans"` rule in `selection.SELECTION_METHODS`. `select_prune_indices_by_method(weight, n, method, **kwargs)` — the single dispatcher `vgg.py` and `oneshot.prune_model` call — now lives in `selection.py` and never imports this module: registered selectors (like `"kmeans"`) are called directly, and every saliency method goes through `compute_score` → `select_prune_indices` exactly as before, so the D1 guarantee (one score, one selection) still holds. A cluster-based rule isn't a per-channel score, which is why it's a selector rather than a scorer.

Extension points: `registry.py`, `selection.py`, `distance.py`, `module_rules.py`, `indices.py`

Added 2026-10-02 (the open/closed refactor; every pruning path was verified bit-identical against the previous implementation — same pruned weights, same `DependencyGraph` groups on a real ResNet-18, same VGG mask/compress result). `registry.Registry` is the one mechanism: a named table with `register` (also a decorator; refuses to shadow an existing name unless `overwrite=True`), `get` (the usual "unknown method 'x', expected one of [...]" error), and `resolve` (a name is looked up, anything else is taken to already be an implementation — which is why `method=` can be a callable and `metric=` a `DistanceMetric`). The registries:

- `saliency.SALIENCY_METHODS` — `(weight, **kw) -> Tensor[out]` scorers.
- `selection.SELECTION_METHODS` — `(weight, prune_amount, **kw) -> prune_idx` selectors for rules that aren't a per-channel score (`"kmeans"`).

Both contracts are checked where they're dispatched: a scorer must return shape `(out,)`, a selector exactly `prune_amount` (clamped to `[0, out]`) distinct, in-range indices — anything else raises, naming the rule. - `distance.DISTANCE_METRICS` — `DistanceMetric(pairwise, centroid)`; the centroid is the K-Means update that minimizes that distance, kept in the same object so the two can't be registered inconsistently. `centroid=None` is fine for scanning; `kmeans` rejects it. - `quantization.QUANTIZATION_METHODS` — same-shape round-trips `quantize_model_` can apply. INT8 is deliberately absent: it returns an `Int8Tensor`, a different contract. - `module_rules.MODULE_RULES` — `ModuleRule`s (`role`, `rebuild`; optional `output_weight`, `mask`, `commit_mask`, `channel_dim`), keyed by class, resolved through the MRO (`registry.resolve_by_type`), so subclasses inherit their parent's rule. The BatchNorm rule is registered on the private bases `_BatchNorm` and `_InstanceNorm`, so `BatchNorm1d/2d/3d`, `SyncBatchNorm` and `InstanceNorm1d/2d/3d` all resolve to it. - `op_rules.OP_PROPAGATORS` — keyed the way `torch.fx` records a node's target: the function for `call_function` (`torch.add`, `torch.cat`, `torch.flatten`), the method name for `call_method` (`"add"`, `"view"`), and the module class for `call_module` (`nn.Flatten`, MRO-resolved like `MODULE_RULES`). Each entry is a `(walk, source, node, idx)` propagator.

`DependencyGraph(..., module_rules={...}, op_propagators={...})` layers per-graph overrides over the global tables (`ChainMap(overrides, REGISTRY.entries())`) without registering anything globally. The global side is a live view, not a copy, so a rule registered globally after the graph was built still reaches it (section 3).

`indices.py` holds the two index helpers that used to be re-derived inline in four or five places each: `complement_indices(n, idx)` ("which of `0..n-1` aren't in `idx`") and `expand_blocks(idx, block_size)` (channel → its block of flattened columns, head → its block of Q/K/V rows). It also sets the device policy: an index set is a CPU `long` tensor (`as_index`) wherever the weights it indexes are, and every indexing site moves it to the tensor's device first (`on_device_of`). Indices used to stay on whichever device made them, which on a GPU failed at the first prune — fixed 2026-10-11, verified on a GPU 2026-10-13 (see section 6). Every factory call that builds an index set names `device="cpu"` itself, so the policy holds under `torch.set_default_device("cuda")` / `with torch.device("cuda"):` too.

`prunelib/transformer/`

Merged 2026-11-12 from the standalone `transformer_pruning` repo (its own `docs/KNOWLEDGE_TRANSFER.md` is the long-form reference for this subpackage; this section is the map). Where core `prunelib` is stateless tensor-in/tensor-out, this works on a live model. Import everything from `prunelib.transformer`; the subpackages are internal layout.

The pipeline is **measure** → **select** → **cut**, with each step behind an interface:

- **Reaching into a model — `models/`.** `FFNLayerAdapter` (get/set an FFN pair) and `AttentionLayerAdapter` (get/set the Q/K/V/O quartet, per-layer head count) are split because no consumer needs both. `BertLayerAdapter` covers BERT and RoBERTa. Nothing names it directly: every consumer calls `resolve_adapter(model)`, which consults a registry (`register_adapter`) and unwraps HF task models via `base_model`, so a `BertForSequenceClassification` can be passed anywhere.
- **Measuring — `measurement/`.** `FFNCalibrator` streams per-neuron activation moments (`CalibrationStats`: mean, mean_abs, rms), and `collect_many` does every layer in one pass. `HeadCalibrator` gives per-head contribution moments (`HeadStats`) plus Michel et al.'s gradient importance `E|dL/dξ_h|`, placing the head gate with a pre-hook (transformers 5.x dropped `head_mask`). `ActivationRecorder` keeps full per-token firing patterns, which twin detection needs. All share `run_hooked`, which always removes its hooks and restores train/eval mode.
- **Deciding — `selection/`.** Selectors get a context, not a bare weight: `FFNContext` carries `W_in`, `W_out`, biases and optional stats, because a neuron's contribution is `W_out[:, i] · act(W_in[i]·x + b_i)`. FFN criteria: `SaliencySelector(scorer)`, `InOutNormSelector`, `ActivationAwareSelector`, `TwinRedundancySelector`, `WeightClusterRedundancySelector`. Head criteria: `RedundantHeadSelector`, `OVNormHeadSelector`, `ActivationAwareHeadSelector`, `GradientHeadSelector`. FFN selectors return a `Selection` (keep list plus optional `merge_map`).
- **Cutting — `surgery/`.** `FFNSurgeon` slices, folds merges and applies bias compensation. `AttentionSurgeon` removes whole heads and can also compensate. It can't merge, because two heads that attend alike can still read and write different subspaces.
- **Orchestrating — `pipeline/`.** `prune_ffn_layer` / `prune_attention_layer` act on one layer; `prune_model_ffn` / `prune_model_attention` act on the whole stack under an `AllocationStrategy`: `"uniform"`, `"global"` (rank every unit model-wide after per-layer normalization, with a `min_keep_ratio` floor), or your own.

Two operational rules to remember. **Cut heads before FFNs, and re-calibrate in between**, because the head cut changes every downstream FFN's input distribution. **Calibration stats go stale after surgery**: the contexts and surgeons check neuron/head counts and raise rather than reuse them silently (use `stats.select(kept)` if you mean to).

Seams with core (enforced by `tests/transformer/test_package_layout.py`: core never imports `prunelib.transformer`): - `CSDScorer` calls `prunelib.csd_saliency`. - `RegisteredScorer("l1")` turns any core-registered method into a `SaliencyScorer`. - `AttentionHeadAnalyzer.compute_distance_matrix` uses `prunelib.pairwise_distance_matrix`. - pandas is optional, imported only inside `NetworkSaliencyScanner.scan()` (the `[analysis]` extra).

`prunelib/evaluate.py`

`count_params`, `count_encoder_params` (params under a named submodule, e.g. `.encoder` or a dotted `"bert.encoder"`, excluding embeddings; it raises if the submodule is missing, where it used to count the whole model), `measure_latency` (wall-clock, warmup + averaged iterations; every submodule's train/eval

mode is restored afterwards; a tuple input is unpacked into positional arguments; a CUDA device is synchronized before each clock reading, since kernels run asynchronously), and `estimate_size_bytes` (`count_params * bits_per_param / 8` — a quick way to compare quantization precisions, not a substitute for measuring a saved checkpoint, which also carries INT8 scale/zero-point metadata). Nothing subtle here, but note: `measure_latency`'s numbers are hardware- and batch-size-dependent. Don't hardcode a specific multiplier anywhere that isn't clearly labeled with the machine/conditions it came from — the README once quoted 1.8x where a later `00_demo.py` run measured 2.13x, which is why its Results section now says to measure on your own machine before citing one.

`experiments/`

Experiments that touch a real pretrained model class (`01`, `02`, `03`, `06`) follow a three-tier pattern. The rest (`00_demo`, `04_coactivation`, `05_ordering`, `07_quantization`) are synthetic end to end, run in seconds with no flag, and CI runs them as-is.

The three tiers:

- **`--smoke`** — fully synthetic data and a tiny hand-built model (no `torchvision`/`transformers` model classes involved for `01`). Runs in under a second. This is what CI runs on every push.
- **`--tiny-check`** (on `01` and `06`) — the *real* model class (`torchvision.models.vgg16` / `resnet18`) with randomly-initialized weights and `FakeData` or random input instead of a real dataset. No network access needed, but it exercises the actual code path `run_full()` uses. Takes minutes, not seconds — not run in CI, but should be run manually after any change to `prune_vgg_layer` or `build_vgg16`.
- **no flag / `run_full()`** — the real thing. Downloads real weights and a real dataset. Needs internet access and has not been executed anywhere yet (see section 6) — everything up to this point has only been verified mechanically.

If you add a new experiment that touches a real pretrained model, follow this same three-tier pattern rather than inventing a new one.

`08_transformer_techniques.py --smoke` runs the `prunelib.transformer` pipeline on a random-init `BertForSequenceClassification`: global gradient-ranked head pruning, re-calibration, global activation-aware FFN pruning (both with bias compensation), then a twin merge. Random weights have no real twins, so it plants one exact duplicate neuron to exercise the merge path. It has no real-run mode; that is what `research/` is for.

`experiments/research/` — the study behind a paper

Generalises `transformer_pruning`'s MRPC harness to GLUE × seeds × criteria × budgets, for free interruptible GPUs. Read `docs/RUNNING_EXPERIMENTS.md` before running it. Four files, one job each:

- `glue.py` — task specs (primary metric per task: accuracy for MRPC/QQP, because majority-positive data makes F1 uninformative), tokenization, sklearn/scipy metrics, and `new_trainer`, which **must** be called fresh after every surgery (a reused Trainer's optimizer holds the replaced parameters and silently trains nothing).
- `variants.py` — the 23 conditions as `name -> apply(model, percent, ctx)`, grouped into presets `rq1-ffn`, `rq1-heads`, `rq2-order`, `quick`. A new condition is one dict entry.
- `run_study.py` — one cached baseline per (task, model, seed). Appends one JSON line per finished run; skips finished runs; `--resume-from` merges a previous session; `--time-budget-hours` stops

cleanly; pins one GPU so Kaggle T4×2 doesn't change the effective batch size; records GPU, versions and git commit on every row. Failed runs go to `errors.jsonl` and are retried next time.

- `summarize.py` — mean ± std over seeds with **paired** drops (each seed against its own baseline), Markdown/CSV tables, and plots.

`--smoke` runs all of it on a random-init tiny BERT with synthetic data in seconds (in CI). It has also been checked on real MRPC/RTE with `prajjwal1/bert-tiny`. Those numbers are mechanics only.

5. Testing conventions

Every test in `tests/` is named after either a historical defect (`test_d1_...`, `test_d6_...`) or a specific behavior guarantee (`test_ffn_block_shrinks_and_preserves_values`). When you fix a bug in this codebase, the workflow is:

1. Write a test that fails against the buggy code, named `test_dN_<what_broke>` if it's a regression of a known historical pattern, or descriptively otherwise.
2. Fix the bug.
3. Confirm the test passes and nothing else broke: `pytest tests/ -v`.

This means the test suite doubles as a defect log — reading `tests/` top to bottom tells you most of this codebase's incident history without needing `GITHUB_AUDIT.md` open at the same time. Keep it that way: don't delete a defect-named test even if you refactor the code it guards, unless you're certain the refactor makes the bug class structurally impossible (as, e.g., switching to `torch.topk` made D2 impossible to reintroduce even accidentally).

Run everything: `PYTHONPATH=. pytest tests/ -v` (178 tests, two of which need a CUDA device and are skipped without one; ~40-45s total on CPU; the pure-`prunelib` tests alone (including `test_graph.py`, `test_quantization.py`, and `test_evaluate.py`, none of which need `torchvision` / `transformers`) are still ~2-3s, the rest is the handful of tests that build a real `torchvision.vgg16` or a real HF BERT model). Those VGG16 tests also need a few GB of free RAM: on a 6.6 GB laptop with other work open, `test_legacy_pipeline.py` was OOM-killed (2026-10-02) while the other ten files passed run on their own. If `pytest` dies with exit code 137, run the files separately before suspecting the code.

The two slowest tests train/evaluate a real (random-init) VGG16 against `archive/legacy_pipeline`'s FakeData path — `PruningConfig.fakedata_train_size` / `fakedata_test_size` (default 32/16, matching the old hardcoded values) exist specifically so a test can shrink the synthetic dataset instead of paying full VGG16-forward-pass cost on every batch. Fixed 2026-09-22: `test_run_pruning_end_to_end_tiny_check` had been silently running at the config default `image_size=224` despite being the "tiny check" — its sibling test already used 32. That one line, plus shrinking both tests' FakeData sizes via the new config fields, cut the full suite from ~59s to ~26-31s without weakening what either test verifies (still 2 distinguishable epochs, still `len(train_loader) > 1`, still a real end-to-end pipeline). If you add a new test that spins up a real VGG16, use these fields rather than hardcoding a smaller `image_size`/dataset size inline.

`tests/transformer/` follows the conventions of the repo it came from instead: tests are named for the behavior they guarantee (e.g. exact mean preservation under bias compensation, merge of identical neurons being output-preserving, gradient importance matching finite differences). They run against `conftest.py::StubBert`, a hand-built module tree with BERT's `encoder.layer[i]` shape, so they need no HuggingFace and run in under a second. `test_package_layout.py` treats the folder layering as architecture and fails on an upward import.

6. Known gaps — read this before claiming something works

Being explicit about what's *not* done is as important as documenting what is, given this project's history of a paper's headline result having no correct public implementation. Current state, honestly:

- ~~**K-Means clustering selection is not in `prunelib`.**~~ **Added 2026-09-30** (`clustering.py`, see section 4), usable via `method="kmeans"` in `prune_model`, `prune_vgg_layer`, `mask_vgg_layer`, and `experiments/01`. Still true: its accuracy numbers (Manhattan's 35.15% params / 49.11% FLOPs on VGG16) haven't been reproduced here, same caveat as `run_full()` below. `pruning_framwork_v4` has its own K-Means implementation; the two haven't been cross-checked against each other on the same weights yet. **SVD and hybrid sequencing are still only in `pruning_framwork_v4`, by design.**
- ~~**`prune_vgg_layer` can't prune the last conv layer**~~ **Fixed 2026-09-30.** `prune_vgg_layer`, `mask_vgg_layer` and `compress_masked_vgg` all resize `classifier[0]`'s input columns to match. `experiments/01`'s `run()` now prunes every conv layer, including the last one. `archive/legacy_pipeline` still skips it (`n_prunable = len(...) - 1`), deliberately, since that package is frozen.
- **`run_full()` in `experiments/01` has never been executed.** It's real code, verified mechanically via `--tiny-check` (real model class, no network), but nobody has run it against actual CIFAR-10 and actual ImageNet-pretrained weights yet. Don't cite a specific accuracy-drop number from this codebase until that's been done.
- **`experiments/02` and `03` are the same situation** — real `transformers` model classes, verified in `--smoke`, never run against a real fine-tuned checkpoint or real SST-2 data.
- ~~**FFN saliency scoring is a reshape hack, not a first-class API.**~~ **Fixed 2026-09-21.** `saliency.py`'s scorers now accept a 2D Linear weight `[out_features, in_features]` natively (via `_channel_view`), alongside the original 4D Conv2d shape. `experiments/02_bert_sst2_sweep.py::_score_ffn_neurons` no longer reshapes around it.
- ~~**No attention-head *pruning* surgery exists**~~ **Fixed 2026-09-21.** `surgery.py` now has `prune_attention_heads`, the equivalent of `prune_conv_bn` for a multi-head attention block (touches Q/K/V rows and the output projection's columns simultaneously). ~~`experiments/03` still only *detects* redundancy~~ **Wired up 2026-09-30:** `experiments/03`'s `prune_redundant_heads` clusters each layer's heads (K-Means/Manhattan on concatenated Q/K/V slices), prunes the lowest-L1 head within each cluster via `prune_attention_heads`, and the smoke run checks the pruned HF BERT still runs a forward pass. Whether weight-space clustering is the *right* redundancy criterion for heads is still open, since it's untested on a fine-tuned checkpoint.
- **`CoActivationScanner`'s 0.9 firing-rate ceiling is a design choice, not a validated threshold.** Only tested against hand-constructed synthetic masks so far.
- ~~**Every surgery function needs the caller to already know the seam.**~~ **Fixed 2026-09-22.** `prunelib.graph.DependencyGraph` traces an arbitrary model with `torch.fx` and works out which other layers a prune decision couples to on its own (residual/skip, squeeze-and-excitation gates, `cat`, the flatten-into-classifier boundary, depthwise convs) — see its module walkthrough entry above and `graph.py`'s own docstring for what's still explicitly out of scope: non-depthwise grouped convs raise rather than attempt, a module called more than once (shared weights) raises, and channels stay on the dimension their producer put them on — a Linear over `[N, T, features]` is followed on its feature dimension since 2026-10-11, but a `permute`/`transpose` into channels-last (ConvNeXt-style blocks) raises, and there is no rule for `LayerNorm` (it normalizes across the channels, so pruning some changes the others' outputs — not a per-channel layer). ~~Only one `cat` branch is safely pruned at a time~~ **Fixed 2026-10-11.**

- ~~**The CUDA path has never run.**~~ **Verified 2026-10-13.** The 2026-10-11 device fix (indices live on the CPU and move to the data's device to index it, K-Means runs on the CPU, `measure_latency` synchronizes a CUDA device) was made by reading the code. The development laptop was thought to have no GPU, but it has a GTX 1650 (4 GB, sm_75); only `venv/`'s `torch` was a CPU build. `venv-cuda/` (torch 2.14.0+cu132, same versions otherwise) runs it. Both CUDA tests pass, the whole suite passes there (178, none skipped), and every experiment now takes `--device` (default `cuda` when available). Each prints the same channel counts, params and scores on `cuda` as on `cpu`, except `04`'s two-decimal firing rates, where float32 rounding flips an exact .5. `01 --tiny-check` (real VGG16 + Adam, batch 8) fits in 4 GB and takes ~21 s. Running every test with CUDA as torch's default device found one more bug, fixed the same day: factory calls that didn't name a device (section 3's index-set row). What's still unverified: real data on a GPU (`experiments/01` full run), and any GPU other than this Turing card.
- ~~`DependencyGraph` **isn't wired into the two-phase mask-then-compress workflow — only one-shot surgery.**~~ **Fixed 2026-09-23.** `PruningGroup.mask()` / `.commit_and_compress()` extend `masking.py`'s two-phase design to a whole dependency group — see the `graph.py` walkthrough entry above.
- ~~**No automatic attention-block discovery in** `DependencyGraph`**, and no hook for one.**~~ **Partially addressed 2026-09-23** — `special_handlers` (see the `graph.py` walkthrough entry above) is a real, tested extension point now, not just a described-but-absent idea. What's still true, and still deliberate: it only fires when the *registered* type is the `get_pruning_group` call's starting layer, not when it's discovered automatically mid-graph or found without the caller naming the type up front — genuine automatic attention-block *discovery* (recognizing a reshape/matmul/softmax/matmul sequence as "one attention head" without being told) is still the "much harder problem" `graph.py`'s module docstring disclaims, and still isn't attempted.

- **Quantization now exists (** `prunelib/quantization.py`**, added 2026-09-23), closing what section 10.2 used to describe as "not present in this codebase at all."** All three thesis Ch. 6.6 methods are implemented and tested (Float16, linear INT8, Fixed-Point32) — see the module walkthrough entry above. What's still a gap, honestly: none of them have been run against a real fine-tuned model to reproduce the thesis's own accuracy-drop numbers (1.34% for Float16, 3.26% for Fixed-Point32 on a pruned VGG16) — `experiments/07_quantization.py` verifies the mechanism (precision loss is real, sizes shrink as expected) on a synthetic model and random data, same caveat section 6 already states for `experiments/01`'s `run_full()`. Don't cite a specific post-quantization accuracy number from this codebase either, for the same reason.

- **No** `prunelib.transformer` **criterion has a reproduced accuracy number in this repo yet** — the harness exists (`experiments/research/`), but it hasn't been run at BERT-base scale. The mechanics are tested (and the source repo reports a check against a real transformers `BertForSequenceClassification` where pruned logits matched zero-gating the same heads to 1e-8 -- not re-run here), but nothing here has been fine-tuned → pruned → healed at publishable scale. Don't cite a criterion comparison from this repo until `docs/RUNNING_EXPERIMENTS.md`'s staged plan has produced it.

- **Two definitions of "Max-k on a Linear layer" coexist.** Core `"max_k"` treats a Linear weight as a 1x1 conv (top-k over each input's one-tap kernel), so on an FFN it equals L1. `prunelib.transformer.TopKMagnitudeScorer(k)` sums each neuron's k strongest incoming connections. They are deliberately not reconciled until the papers' intended definition is confirmed. Pick one explicitly in any experiment, and note that `RegisteredScorer("max_k")` gives the core (L1-equivalent) behavior.

- **Two `prune_attention_heads` functions.** `prunelib.prune_attention_heads(q, k, v, o, keep_heads, num_heads)` is tensor-level and takes heads to **keep**. `prunelib.transformer.prune_attention_heads(model, layer, head_indices)` is model-level, takes heads to **drop**, and updates the layer's head-count bookkeeping. Check which one you imported.

- **Two k-means implementations.** `prunelib.kmeans` (K-means++ seeding, per-metric centroids, empty-cluster repair; drives `method="kmeans"`) and `prunelib.transformer.kmeans_assign` (plain Lloyd's, used by `WeightClusterRedundancySelector` on L2-normalized FFN rows). Switching the selector to `prunelib.kmeans(..., metric="cosine")` would unify them but changes which neurons it keeps -- do it before the GLUE study produces numbers, not after.

- **`CoActivationScanner` and `prunelib.transformer.JaccardTwinFinder` overlap** (both Jaccard over firing patterns; the latter also reports dead neurons and works from real HF activations via `ActivationRecorder`). Not consolidated.

7. Extending this codebase — where new work goes

- **New scoring method** (e.g. a Taylor or activation-based saliency for Transformer FFN neurons — a new *CNN* criterion goes to `pruning_framwork_v4`, see below): write a function matching the existing signature (`weight` — any `[out, in, *kernel]`: Conv2d `[out,in,kh,kw]`, Linear `[out,in]`, ... — `-> Tensor[out]`; reshape through `saliency._channel_view` rather than assuming a rank) and decorate it with `@register_saliency_method("name")` — in `saliency.py` if it belongs in the library, or in your own code if it doesn't; either way `compute_score`, `select_prune_indices_by_method`, `prune_model` and `vgg.py` accept it with no further edit. Add a test in `tests/test_saliency.py` following the existing pattern (independent brute-force comparison where feasible, like `test_d2_matches_bruteforce_topk`).
- **New surgery type** (`prune_attention_heads` is the worked example now): add to `surgery.py`, build it from the `slice_*` primitives (don't construct modules and copy weights by hand — that's how three diverging copies of the same logic happened), validate the seam and raise before mutating anything (follow `prune_ffn_block`'s pattern exactly), add a test that checks *values* survive correctly post-surgery, not just shapes (see `test_d6_conv_values_are_correct_not_just_shape` for why shape-only tests aren't enough — that's literally how D6 shipped originally), and where feasible verify against a real HF model forward pass, not just plain `nn.Linear` (see `test_prune_attention_heads_against_a_real_hf_bert_model`).
- **New experiment**: follow the three-tier `--smoke` / `--tiny-check` / full pattern from section 4. Add the `--smoke` invocation to `.github/workflows/tests.yml` so CI actually exercises it.
- **Extending `prunelib.graph.DependencyGraph`**: `special_handlers` (`DependencyGraph(..., special_handlers={SomeAttentionClass: your_handler})`, paired with `LeafTracer([SomeAttentionClass])` so the traced graph actually sees that class as one node) is a real, tested attention-block delegation point now — see the `graph.py` walkthrough entry in section 4 and `test_graph.py::test_special_handler_prunes_attention_block_via_prune_attention_heads` for the worked example. It's scoped to the *starting* layer of a `get_pruning_group` call, not to automatic discovery mid-graph — if you want a specific attention class recognized as a prunable starting point without the caller naming it, that's the actual remaining gap, not the handler mechanism itself. `PruningGroup.mask()` / `.commit_and_compress()` (also added 2026-09-23) already wire `DependencyGraph`-based pruning into the two-phase mask-then-compress workflow — that's no longer open either. To make `DependencyGraph` prune *through* a new layer type (Conv1d, GroupNorm, ...), don't edit `graph.py`: write a `module_rules.ModuleRule` (`role` + `rebuild`, plus `output_weight`

and `mask` / `commit_mask` if its output channels aren't dim 0 of its weight — the default `mask` refuses a rule that overrides `output_weight` alone — and `channel_dim` if its tensors don't keep channels on dim 1) and register it with `register_module_rule`, or pass `module_rules={Type: rule}` to one graph; `prune_model` then scores and prunes that type too. Until a type has a rule, a module of it that owns any parameter or buffer stops the walk with a message naming both fixes. `tests/test_extension_points.py::test_module_rule_teaches_dependency_graph_a_new_layer_type` (Conv1d) and `test_module_rule_says_where_a_layers_output_channels_are` (ConvTranspose2d) are the worked examples. To make it propagate through a new *op* (a residual merged with `torch.maximum`, say, or an activation function of your own), don't edit `graph.py` either: `register_op_propagator(torch.maximum, propagate_elementwise)` (and `"maximum"` for the tensor method), or `register_op_propagator(my_activation, propagate_channelwise)`, or `op_propagators={...}` on one graph; a genuinely new op gets its own `(walk, source, node, idx)` function. Read where the channels are with `walk.channel_dim(source)` (counted from the end), never dim 1; pass `dim=` to `walk.forward` if the op moves them; and use `walk.forward_part` instead of `walk.forward` if the op's output loses channels from several inputs at once, as `propagate_cat` does. `test_extension_points.py::test_op_propagator_teaches_dependency_graph_a_new_op` and `test_channelwise_propagator_teaches_dependency_graph_a_functional_op` are the worked examples.

- **New quantization method** (e.g. logarithmic or k-means-based quantization, which the thesis flags as better fits for skewed weight distributions than linear INT8 — see section 10.2): add a function to `quantization.py` matching the existing per-tensor signature (`Tensor -> Tensor`, or a small dataclass like `Int8Tensor` if it needs auxiliary metadata to dequantize), add a test following `test_quantization.py`'s pattern (brute-force comparison against the method's own formula where feasible, like `test_int8_quantize_matches_the_documented_formula`; a clipping/wrapping-at-the-boundary test if the format has a bounded range, like `test_fixed_point32_clips_out_of_range_values_instead_of_wrapping`). If it round-trips to a same-shape, same-dtype-castable tensor, also decorate it with `@register_quantization_method("name")` so `quantize_model_` can apply it; if it needs a struct like INT8 does, don't register it (it would break `quantize_model_`'s uniform copy-back loop) — document why it's excluded instead, the way `quantize_model_`'s own docstring does for INT8.
- **A new cluster-based selection rule** (e.g. a different within-cluster tiebreak than L1, or agglomerative clustering): add it to `clustering.py` with `@register_selection_method("name")` — signature `(weight, prune_amount, **kwargs) -> prune_idx` — so every caller gets it without growing its own branch (and without `selection.py` importing it). A new distance metric for it goes through `register_distance_metric`, with the centroid rule that minimizes it.
- **New Transformer criterion**: subclass `ImportanceSelector` (FFN) or `HeadImportanceSelector` (heads) and implement `importance(ctx)`. That alone makes it work with uniform *and* global allocation. Need more than a per-unit score (merges, structural rules)? Subclass `NeuronSelector` / `HeadSelector` and override `select`, and accept that global allocation will reject it. Pure weight scores can also just be core methods (`register_saliency`) wrapped in `RegisteredScorer`. Tests go in `tests/transformer/`, against `StubBert`.
- **New model family** (GPT-2, T5, ViT, ...): write an adapter implementing `FFNLayerAdapter` and/or `AttentionLayerAdapter`, and call `register_adapter(matches, factory)` once. No consumer changes.
- **New allocation policy**: subclass `AllocationStrategy` and pass it as `allocation=`. It sees neurons and heads through the same `PrunableStructure` interface, so one class serves both.
- **A new CNN pruning criterion (SVD, second-order, anything matching the papers other than K-Means) belongs in `pruning_framwork_v4`, not here.** That repo is canonical for CNN methods and

already has more coverage than this package's `prunelib`. Adding one here would create a second, divergent implementation of the same idea — see the scope note at the top of section 1.

- **Transformer-extension work** (`experiments/03` already prunes heads via K-Means on Q/K/V weights and `prune_attention_heads`, since 2026-09-30; what's open is testing whether weight-space clustering is the right head criterion on a fine-tuned checkpoint, and validating `CoActivationScanner`'s threshold against real activations) is this repo's actual remaining job — see the gaps list in section 6 for what's still open there. `transformer_pruning` has been merged in as `prunelib.transformer`, so do new Transformer work here -- changes to the standalone repo's copy would diverge from this one.

8. Onboarding checklist

First: check the scope note at the top of section 1. If you came here to add or fix a CNN pruning criterion, you probably want `pruning_framwork_v4` instead.

```
git clone https://github.com/thakerpragnesh/structured-pruning.git
cd structured-pruning
pip install -e ".[dev,vision-experiments,transformer-experiments,research]"
pytest tests/ -v                        # 390 tests (3 skipped without a GPU)
python experiments/00_demo.py           # full pipeline, seconds
python experiments/01_vgg_cifar10_sweep.py --smoke
python experiments/06_generic_pruning.py --smoke
python experiments/07_quantization.py   # pruning + all three quantization methods
python experiments/08_transformer_techniques.py --smoke   # prunelib.transformer end to end
python experiments/research/run_study.py --smoke --variants all   # the GLUE study harness
```

On a GPU, install a CUDA build of `torch` before the package (match the CUDA version to what `nvidia-smi` reports). The development laptop's `venv-cuda/` was built this way:

```
python3 -m venv venv-cuda
TMPDIR=~/.cache/pip-tmp venv-cuda/bin/pip install torch==2.14.0 torchvision==0.29.0 \
    --index-url https://download.pytorch.org/whl/cu132
venv-cuda/bin/pip install -e ".[dev,vision-experiments,transformer-experiments]"
venv-cuda/bin/python experiments/00_demo.py   # prints "device: cuda"
```

`TMPDIR` matters on that laptop. `/tmp` is a 3.4 GB tmpfs, and pip unpacks the ~4 GB of CUDA wheels there, so without it the install fails with `Disk quota exceeded`. Colab and Kaggle: `notebooks/run_on_colab_kaggle.ipynb` (CNN pipeline) and `notebooks/pruning_study_kaggle_colab.ipynb` (the GLUE transformer study -- read `docs/RUNNING_EXPERIMENTS.md` first).

Read section 3 of this document before touching `prunelib/saliency.py`, `prunelib/surgery.py`, `prunelib/masking.py`, `prunelib/selection.py`, or `prunelib/graph.py` and its neighbours (`group.py`, `op_rules.py`), and section 4's "Extension points" entry before adding a method, metric, layer type or op (it's usually a `register_*` call, not an edit). Read section 6 before writing anything that implies full VGG16/CIFAR-10 results or BERT fine-tuning results are already produced by this codebase — the mask-then- compress *mechanism* is tested and correct; specific accuracy numbers from real training runs haven't been produced here yet.

9. Glossary

- **Saliency score** — a per-channel (or per-neuron) number estimating how much that channel contributes to the model's output; lower means "safer to prune."
- **Structural surgery** — physically resizing a layer (fewer real rows/ columns in a real tensor) as opposed to zero-masking, which keeps the tensor the same size and just sets some entries to zero. Surgery makes the model smaller and faster; masking only makes it sparser.
- **Seam** — the shape contract between two connected layers (a conv's output channels matching the next conv's input channels, or an FFN's `fc1.out_features` matching `fc2.in_features`). "Seam mismatch" is this codebase's term for what happens when surgery on one layer isn't propagated to the layer downstream of it.
- **FFN block** — the two-Linear-layer (expand, then project back down) feed-forward sub-layer inside each Transformer block.
- **Co-activation** — two units (neurons, heads) firing on the same inputs; a redundancy signal independent of whether their weights look similar.
- **Firing rate** — the fraction of tokens/inputs a given unit is "active" on, used to exclude near-universal-firing units from co-activation analysis (see section 4).
- **Mask-then-compress** — the two-phase pruning design in `masking.py`: zero a channel's contribution via reparametrization first (reversible, same tensor shapes, safe to fine-tune against), and only physically resize the model once, at the end, via `compress_masked_conv_bn`. As opposed to one-shot surgery (`prune_conv_bn` called directly), which resizes immediately on every pruning decision.
- **Scorer vs. selector** — a scorer (`SALIENCY_METHODS`) gives every channel its own number and the lowest are pruned; a selector (`SELECTION_METHODS`) returns the indices to prune directly, for rules where a channel's fate depends on the others (K-Means). Both are valid `method=` values.
- **Registry** — a `registry.Registry`: a public name (or type) → implementation table that a `register_*` call extends from outside `prunelib`, so adding a method never means editing the library.
- **Channel role** — a `ModuleRule`'s answer for one layer type: `MIXING` (ungrouped Conv2d, Linear) produces its own channels, so a prune stops at its input; `PER_CHANNEL` (BatchNorm, depthwise conv) shrinks and passes the prune through.
- **Op propagator** — the function `DependencyGraph` calls to carry a prune through an op that owns no weights (add, cat, flatten); registered in `op_rules.OP_PROPAGATORS`.
- **Pruning group** — every module one prune decision forces to resize, as `DependencyGraph.get_pruning_group` returns it (`PruningGroup`), ready for `.prune()` or `.mask()` / `.commit_and_compress()`.
- **Calibration** — running a small set of real inputs through the model with hooks attached, to measure how units actually behave (activation moments, head contributions, gradient importance), as opposed to judging them from weights alone.
- **Bias compensation** — when a unit is removed, adding its *mean* output contribution (`W_out[:, i]·E[a_i]`, or `W_O[:, h]·E[ctx_h]` for a head) to the next layer's bias. Turns zero-ablation into mean-ablation; the layer's mean output is preserved exactly on the calibration distribution.
- **Merge** — folding a removed FFN neuron's output column into a surviving "twin" (scaled), so its signal is carried rather than deleted.
- **OV circuit** — a head's `W_O[:, h] · W_V[h]`: the map from what it attends to onto what it writes into the residual stream. Its norm bounds how much the head can matter.

- **Uniform vs. global allocation** — uniform cuts every layer by the same fraction; global ranks every unit in the model together (after per-layer score normalization), so redundant layers give up more.
- **Adapter** — the object that knows one model family's layout (`encoder.layer[i].intermediate.dense`, ...), so nothing else in `prunelib.transformer` has to.

10. Thesis material not reflected in this codebase

Pragnesh Thaker's Ph.D. thesis, *Pruning and Quantization Techniques for Deep Neural Network Acceleration* (NITK Surathkal, July 2025), is the primary source behind this repo — `prunelib`'s Max-k saliency, K-Means/distance-metric selection and scanning, and `experiments/05`'s ordering question (the thesis's hybrid pruning) all trace back to it, and the numbers in `README.md` and `PUBLICATIONS.md` are drawn from it. This section covers what the thesis describes that **isn't** implemented or documented anywhere in this codebase: methods `prunelib` doesn't have (SVD pruning, kernel-level pruning, FC-neuron pruning, custom regularization), the original framework design the current code replaced, and results (external benchmark comparison, future work) not summarized elsewhere in this repo's docs. (Quantization was on this list until 2026-09-23; section 10.2 now records how it was implemented.) General ML background from the thesis — CNN architecture history (LeNet through ResNet), optimizer survey (SGD through Adam), training techniques (dropout, batch norm, cyclic LR), and the literature review of other authors' pruning papers — is deliberately left out here as generic textbook content rather than codebase-specific knowledge.

10.1 Pruning methods in the thesis that `prunelib` doesn't have

SVD channel pruning (thesis Ch. 4.5) — condenses each output channel's kernel into a single scalar via a bit-weighted sum (`Condense_Value = Σ 2^(k1×3+k2) · W[i][j]`, thesis Eq. 4.10), runs Singular Value Decomposition across channels, and uses the singular-value magnitude as the saliency score — channels with the smallest singular values are pruned. This is a genuinely different saliency signal from `saliency.py`'s L1/L2/Max-k (which all operate directly on raw weight magnitudes, never touch SVD). Thesis result (Table 4.12, VGG16/CIFAR10): 26.19% FLOPs / 36.49% parameter reduction at a 1.31% accuracy drop — weaker than Max-k but still usable. Not present anywhere in `prunelib`.

Kernel-level pruning (thesis Ch. 4.4, Algorithm 4.4/4.5, and Ch. 5.3 for the similarity variant) — masks individual *kernels* within a channel (the `[in_channel, kh, kw]` slice for one output channel) rather than the whole channel. Because each kernel in a channel operates on a different input channel and their outputs are summed, kernels can't simply be deleted and the tensor reshaped the way channels can — the thesis's approach is to zero the kernel's weights and skip it at compute time via a data selector, so the *parameter count* doesn't shrink but FLOPs do. Thesis results (VGG16/CIFAR10): saliency-based kernel pruning reached 27.74% FLOPs reduction at a 0.96% accuracy drop (Table 4.6) before the 4th iteration exceeded threshold; the similarity-based kernel variant (Table 5.6) was markedly worse (19.57% FLOPs at a 1.49% drop — the thesis's own conclusion, Ch. 5.3.1, is that grouping kernels by cross-channel similarity "is not showing fruitful results" since kernels on different input channels legitimately encode different information). Neither the masking-based kernel pruning workflow nor a kernel-level saliency/similarity score exists in `prunelib.saliency` or `prunelib.surgery` — both only operate at whole-channel (or whole-head, for attention) granularity.

Fully-connected / neuron pruning (thesis Ch. 6.5) — prunes neurons out of a classifier's dense layers using an L1 saliency score per neuron: the sum of the absolute values of that neuron's incoming *and* outgoing connection weights (not just one side, which is what makes this different from a naive per-layer L1 prune). Applied to VGG16's classifier head, 5% of neurons were removed per fully-connected layer per iteration, for 5 iterations, within a 1% accuracy-drop budget. `prunelib.surgery.prune_ffn_block` could physically

implement the resulting surgery (it already shrinks a Linear → activation → Linear pair), but nothing in `prunelib` computes this specific incoming+outgoing neuron saliency score, and `vgg.py` never prunes `model.classifier`'s neurons (VGG's 3 dense layers) — it only resizes `classifier[0]`'s *input* columns to follow a pruned last conv layer (2026-09-30), which removes no neuron from the classifier itself.

Custom regularization (CSD — Custom Standard Deviation) (thesis Ch. 6.1, Eq. 6.1–6.4; this is `PUBLICATIONS.md` paper #2) — a training-time group-regularization loss, not a pruning-time surgery step, so it's a different kind of gap: `prunelib` has no training loop at all (that lived in `archive/legacy_pipeline/train.py`, and even that has no regularization term beyond a generic L1 penalty). The CSD loss for channel *i* in layer *l* is the ratio of the channel's L1 norm to a per-channel "custom standard deviation" — sum of absolute deviations from the channel's mean weight, not the usual squared-deviation variance:

```
NC_i^l  = Σ_j Σ_k1 Σ_k2 |W[i,j,k1,k2]|
CSD_i^l = Σ_j Σ_k1 Σ_k2 |W[i,j,k1,k2] − Mean|
RegularizationLoss^l = Σ_i (NC_i^l / CSD_i^l)
```

A channel with low internal weight variation (small `CSD`) gets a *large* regularization term relative to its norm, pushing it toward zero faster — i.e. the loss is designed to penalize channels whose weights are already nearly redundant with each other, not just small in magnitude. Fine-tuning with this loss before saliency-based pruning improved VGG16's L1-pruning result from 53.80%/39.04% (no regularization) to 61.91%/46.14% FLOPs/parameter reduction at a comparable ~0.95% accuracy drop (thesis Table 6.4). Nothing in `prunelib` or `archive/legacy_pipeline` implements this loss.

10.2 Quantization — now implemented in `prunelib/quantization.py` (2026-09-23)

The thesis title is "Pruning *and Quantization* Techniques," and Ch. 6.6 covers three quantization approaches applied *after* pruning, as a separate compression stage. All three are now in `prunelib/quantization.py` — see the module walkthrough entry in section 4 for the API, and section 6 for what's still an honest gap (none have been run against a real fine-tuned model to reproduce the thesis's own accuracy-drop numbers below). `archive/legacy_pipeline` and the pre-existing experiments (00 through 06) are unaffected; this repo's scope is otherwise still pruning-focused, with quantization as a separate, final stage you apply to whatever model pruning produced.

Float32 → Float16 (`quantize_float16` / `dequantize_float16`). A direct cast of every weight from 32-bit to 16-bit floating point (1 sign bit, 5 exponent bits, 10 mantissa bits — range ±10^±5, 3–4 decimal digits of precision). Applied to the final hybrid-pruned VGG16 model, the thesis measured this as halving memory footprint for a 1.34% accuracy drop (Ch. 6.6.4) — the thesis reports this as a good trade since AI accelerators and mobile processors widely support Float16 inference natively.

Float32 → Fixed-Point32 (`quantize_fixed_point32`). Parameters are cast into a fixed 1-sign-bit / 3-integer-bit / 28-fractional-bit format; any value outside the representable range is clipped rather than wrapped (`test_fixed_point32_clips_out_of_range_values_instead_of_wrapping` pins this specifically). The thesis measured this as markedly worse than Float16 — a 3.26% accuracy drop on the same pruned VGG16 — because, unlike floating point, a fixed-point format can't dynamically rescale its exponent to fit the actual distribution of a layer's weights, so precision is wasted or values are lost at the tails. One simplification from the thesis worth flagging: the thesis describes a per-layer weight *normalization* step before the fixed-point cast; `quantize_fixed_point32` here is the cast only (round to the nearest representable value, then clip) with no normalization step — closer to the "cast, don't rescale" comparison the thesis draws against Float16 than a full reproduction of Ch. 6.6.2's exact procedure. If you need the normalization step too, it belongs as a

separate, composable function (normalize, then call `quantize_fixed_point32` on the result), not folded into this one.

Linear INT8 quantization (`quantize_int8_linear` / `dequantize_int8_linear`, Eq. 6.7–6.10) — the standard scale/zero-point affine mapping, which the thesis includes as a reference technique rather than one it ran end-to-end on the pruned VGG16:

```
scale = (r_max − r_min) / (q_max − q_min)
zero_point = round(q_min − r_min / scale)
x_int8 = round(x_fp32 / scale) + zero_point
```

with dequantization `x_fp32 ≈ scale × (x_int8 − zero_point)`, and `[q_min, q_max] = [−128, 127]` for signed INT8 — `test_int8_quantize_matches_the_documented_formula` is an independent brute-force check against exactly this formula, in the style of `saliency.py`'s brute-force tests. The one addition beyond the thesis's formula: a constant tensor (`r_max == r_min`) would divide by zero in it, so that case is handled separately (see `quantize_int8_linear`'s own docstring for how). The thesis notes the general formula assumes a roughly uniform weight distribution and flags logarithmic or k-means-based quantization as better fits for skewed distributions — neither is implemented here; see section 7 if you want to add one.

`evaluate.estimate_size_bytes(module, bits_per_param)` is the bit-width/ precision helper this section used to say `evaluate.py` was the natural home for, now added alongside `count_params` and `measure_latency`.

10.3 The original four-module framework (thesis Ch. 3) — useful mainly as the defect record's primary source

The thesis formalizes, in pseudocode, the exact framework `GITHUB_AUDIT.md` and this document's section 1/3 describe from the outside. It's organized as four modules — **Load Model** (save/load/create-custom-model), **Initialize Pruning** (convolution-layer indexing, prune-count-list generation), **Facilitate Pruning** (mask matrix creation, apply-pruning, deep-copy-non-zero-channels), **Train Model** (fit-one-cycle, freeze-model, freeze-selected-layer) — and this codebase's `prunelib` is a from-scratch, corrected reimplementation of the same four responsibilities, not a refactor of this code.

Two pieces of this pseudocode are worth knowing about specifically because they *are* the bugs section 3 documents, now given formal shape:

- **Algorithm 3.9 ("Pruning Algorithm Class")** subclasses `torch.nn.utils.prune.BasePruningMethod` and calls `PruningAlgorithm.apply(module, name)` to mask channels. This is the exact design `masking.py`'s docstring calls out as the root cause of the `layer_number`-global bug (a `compute_mask` callback reconstructing a mask from global state on every call) — `prunelib.masking.mask_channels` replaced this whole pattern with one `torch.nn.utils.prune.custom_from_mask` call using a mask computed once, up front.
- **Algorithm 3.10 ("Deep Model Copy Channel wise")** is, almost verbatim, defect D6: it iterates output channels, checks `norm of source channel parameters is not zero`, and copies surviving channels into the compressed model at a running `out_ch_new` counter that increments only inside the `if not zero` branch — the exact hand-maintained destination-index pattern `surgery.py`'s module docstring says caused the original `deep_model_copy_channelwise` bug (the original never incremented, so every survivor overwrote index 0). Seeing the thesis's own pseudocode for this algorithm confirms this document's account of D6 is describing a real, specified algorithm, not just a colloquial retelling.

The framework's **prune-count-list algorithm** (thesis Algorithm 3.7) also formalizes something `prunelib` does differently on purpose: the thesis schedules pruning per *block* (VGG16's `[2,2,3,3,3]` conv-layer grouping) with a linearly increasing prune probability across blocks (deeper blocks get pruned more aggressively, on the stated grounds that they carry more redundancy) — `prunelib.vgg.mask_vgg_layer` instead takes an explicit `prune_fraction` per call and applies it uniformly to whichever layer the caller names, with no built-in per-block schedule. Anyone wanting to reproduce the thesis's exact schedule in this codebase would need to compute that per-layer fraction list themselves and drive `mask_vgg_layer` with it in a loop — nothing in `prunelib` generates it automatically.

10.4 External benchmark comparison, and the thesis's own future work

Head-to-head with published pruning methods (thesis Table 6.12, VGG16/CIFAR10). `README.md` and `PUBLICATIONS.md` report this thesis's own numbers, but not how they stack up against other published pruning techniques — the thesis includes that comparison directly:

Method	Baseline acc.	Pruned acc.	Acc. drop	Param ↓	FLOPs ↓
SSS (2018)	93.96%	93.02%	0.94%	73.8%	41.6%
GAL (2019)	93.96%	93.77%	0.19%	77.6%	39.6%
HRank (2020)	93.96%	93.43%	0.53%	82.9%	53.5%
CHIP (2022)	93.96%	93.86%	0.10%	81.6%	58.1%
Max-3 (this thesis)	93.28%	92.37%	0.97%	46.14%	61.91%
K-Means (this thesis)	93.28%	92.29%	0.99%	40.00%	40.00%
Hybrid (this thesis)	93.28%	92.47%	0.91%	35.00%	58.34%

The thesis's own reading of this table: SSS/GAL/HRank/CHIP remove a much larger fraction of *parameters* (73–83%) than this thesis's methods do, but this thesis's Max-3 and Hybrid methods reach comparable or better *FLOPs* reduction (58–62% vs. 40–58%) with a similar accuracy drop — i.e. the methods here are relatively more effective at cutting compute than at cutting raw parameter count, which tracks with Max-3/Hybrid being channel-pruning methods concentrated on convolutional layers (per Ch. 1.5, convs are ~5% of VGG16's parameters but ~98% of its FLOPs). None of this comparison table, or the SSS/GAL/HRank/CHIP citations, appear anywhere in this repo.

The thesis's stated future work (Ch. 7.2), distinct from this document's section 6 ("Known gaps") and section 7 ("Extending this codebase"), which describe gaps in the *code*, not the author's own research roadmap:

- Improving SVD-based reconstruction precision/stability post-pruning, to reduce retraining cost while preserving accuracy.
- More robust, possibly learned, evaluation metrics for channel similarity and saliency quality — the thesis notes existing metrics (L1 norm, distance metrics) may not fully capture a channel's functional redundancy.
- Extending pruning/quantization beyond CNNs to transformer architectures and task-specific models (medical imaging, autonomous driving, edge deployment).
- **Adaptive hybrid pruning** that dynamically balances saliency-based vs. similarity-based pruning during training/fine-tuning based on model complexity, data distribution, and hardware constraints — a more automated version of what Algorithm 6.1 (the hybrid-pruning loop, thesis Ch. 6.3) does by hand today.

- Hardware-aware optimization: pruning/quantization decisions driven by target-device characteristics (memory access patterns, parallelism, precision support) rather than architecture-agnostic thresholds.
- Mixed-precision and quantization-aware training, as a follow-on to the post-training quantization covered in section 10.2 above.

This roadmap is worth reading before proposing new work in either `structured-pruning` or `transformer_pruning` (per section 7) — it's the author's own sense of what's still open, independent of what's specifically unfinished in this particular codebase.